Wicket — Product Guide

Generic JSON RPC over Microsoft Dynamics 365 Business Central, built for LLM agents.

PRODUCT	Wicket 1.3 (publisher Dave The Wicketkeeper) with the optional companion `Wicket Actions - Base` 0.3 and the `wicket-mcp` server 2.4
PLATFORM	Business Central 26 and later, SaaS (online). AL runtime 16.0. The core app depends on the System Application only.
GUIDE VERSION	2026-10-28
AUDIENCE	Part I is for everyone. Parts II and III are for the Business Central administrator or consultant who installs and configures Wicket. Parts IV to VI are for the developer or agent builder who calls it.

This guide is the complete manual. It explains what Wicket is, how to install and secure it, how to take the first calls, and how to use every operation, with worked configuration examples for the deployments people actually run. The runtime is always the final authority: `capabilities` and `describe` report what a given tenant exposes right now, and when this text and the endpoint disagree, the endpoint is right.

PART I — UNDERSTANDING WICKET

1. What it is

Business Central ships a REST API with one page per entity. That model works for a fixed integration, but an LLM agent does not know in advance which of the several thousand tables it will need, and nobody can publish an API page for each of them. Wicket replaces the per-entity surface with **one API page and eighteen generic operations**. A caller addresses any table by its number and any enrolled business procedure by codeunit and name. The schema is discoverable through the protocol itself, so an agent can work against a tenant it has never seen: it asks what exists, asks what a table looks like, then reads or writes.

The whole product is three parts:

- **Wicket**, the core Business Central extension. One API page, eighteen bound actions, the deny-by-default security model, the audit log, the change history, telemetry, async export to Azure Blob Storage. It depends on Microsoft's System Application only, so it installs on any Business Central 26 SaaS environment without touching the Base Application.
- **Wicket Actions - Base**, an optional companion extension. It supplies the strongly typed business actions a System-Application-only core cannot own: posting sales and purchase documents, posting general and item journal batches. It runs entirely through Wicket's invoke plane and installs with every action switched off.
- **wicket-mcp**, a Model Context Protocol server (TypeScript, shipped as a self-contained program for Windows and Apple-silicon Macs). It fronts Wicket over stdio so Claude Desktop, Claude Code and any other MCP host see the operations as native tools, handles OAuth for one or many environments, and ships the protocol guide as a tool of its own. Agents without MCP can call the endpoint directly with any HTTP client.

Design principles

- **Deny by default.** Nothing is readable, writable or executable until an administrator exposes it. Fresh installs also hide or mask fields by data classification.
- **Every call answers HTTP 200.** The logical outcome lives in the body: a result object, or an RFC 7807 problem document with a stable error slug and a plain-English fix.
- **Safe to retry.** Optimistic concurrency through an opaque `@etag`, and an idempotency key that replays the original response instead of executing twice (optional for most operations; required for `invoke` and `action` on a fresh install).
- **Audited.** Every call, success or failure, writes one Operation Log row and one Application Insights event.
- **Business Central permissions still apply.** Wicket never widens what the calling principal may do in Business Central; it only narrows it further.

2. How a request flows

```
Agent / MCP server
   │  POST .../companies({companyId})/wicketEndpoint(0)/Microsoft.NAV.{operation}
   │  Authorization: Bearer <Entra ID token>
   ▼
page "Bcmag Rpc Endpoint"            one [ServiceEnabled] bound action per operation
   ▼
codeunit "Bcmag Rpc Dispatcher"      envelope validation · key rule · idempotency replay and
   │                                 reservation · async fork · audit row · telemetry · always 200
   ▼  Codeunit.Run boundary (an AL error becomes a 500 problem; failed writes roll back)
codeunit "Bcmag Op Runner"           gates (Read-Only Mode, then the role rung), then one handler:
   ├─ discover · describe · query (+aggregate) · readBlob · capabilities · skill · pageUrl · whoAmI
   ├─ insert · modify · delete       Value Coercer + Record Access (etag, security filter)
   ├─ invoke · action                Procedure Catalog / Action Catalog → wrapper codeunit
   └─ status · cancel · requests     Request Queue (async delivery)
```

Every field a handler touches passes through the **field policy**; every table passes through **exposure**; every record re-fetch runs under the caller's **security filter**. A `query` that carries a `deliver` block never runs inline: the dispatcher writes a durable queue row and a background worker produces the package later.

The gates, in order

A request meets these checks in this sequence. Each one can refuse independently; nothing later can re-open an earlier refusal.

#	GATE	WHO CONTROLS IT	REFUSAL
1	Entra ID token and a matching identity in Business Central	Azure administrator, BC administrator	HTTP 401
2	Idempotency key for `invoke` and `action` (Setup **Require Key For Actions**)	Wicket administrator	`400 idempotency-key-required`
3	Read-Only Mode (tenant-wide switch)	Wicket administrator	`403 read-only`
4	Wicket role on the principal (the role-gate tables of the permission-set ladder)	BC administrator	`403 role-forbidden`, naming the required permission set. A principal with no Wicket role at all gets HTTP 403 on the endpoint itself.
5	Exposure list and Mode	Wicket administrator	`403 not-exposed`
6	Field policy	Wicket administrator	`403 field-restricted` (hidden), `403 field-masked` (masked field used as a filter, sort or aggregate), `400 masked-value` (write-back of a mask)
7	Procedure or Action Catalog enrolment	Wicket administrator	`404 unknown-procedure`, `403 invoke-disabled`, `404 unknown-action`, `403 action-disabled`
8	Trigger-bypass policy for `init: false` / `validate: false` (Setup **Allow Trigger Bypass**)	Wicket administrator	`403 trigger-bypass-disabled`
9	BC table permissions and validation at write time	BC administrator, business logic	`422 validation`, `422 insert-failed` and similar

Six of these refusals are raised before anything executes and can be lifted by an administrator: `read-only`, `role-forbidden`, `not-exposed`, `invoke-disabled`, `action-disabled` and `trigger-bypass-disabled` (async delivery adds `async-not-configured` and `too-many-requests`). When the request carried an idempotency key, such a refusal is **not** stored as the key's outcome: the key is released, so once the gate is lifted the caller resends with the same key (chapter 18). Every other 4xx, `field-restricted` and the 404s included, is final for its key.

Identifiers you will see

KEY	WHAT IT IS	USE IT FOR
`@recordId`	Opaque token for one row (base64 of the BC RecordId). Embeds the primary key values.	`readBlob`, `modify`, `delete`, `pageUrl`
`@systemId`	The record's stable GUID (`SystemId`). Does not embed the primary key.	`action` targets, `pageUrl`
`@etag`	Concurrency token, by default derived from `SystemModifiedAt`; with Strong ETag on, a keyed hash of the row's values is folded in. Opaque either way.	`modify` and `delete` via `data.etag`
`@masked`	Present on a row only when it carries masked members: the names of the fields whose values are mask renderings, not stored values.	Telling `****45` from a real value; chapter 11
`@requestId`	The envelope `id` echoed back. Problems carry it as `instance: urn:wicket:request:<id>`.	Correlating with the Operation Log
`@nextLink`	Opaque paging cursor.	The next `query` page via `data.cursor`

3. The eighteen operations

PLANE	OPERATION	PURPOSE
Read	`discover`	List reachable tables, invoke-enrolled codeunits, or companies.
Read	`describe`	Full schema of one table (fields, keys, primary key, permissions, per-field readable/writable) or the enrolled procedures of one codeunit.
Read	`query`	Read rows with select, filter, orderBy, top, paging, count, FlowFields, an incremental-sync window, server-side aggregates, and optional async delivery.
Read	`readBlob`	One binary field (BLOB or Media) as base64 with a content type.
Read	`capabilities`	Machine-readable self-description: URL template, transport contract, operation schemas, filter grammar, live limits, posture, enabled actions.
Read	`skill`	Serve the protocol usage guide from a configured HTTPS URL, whole or section by section.
Read	`pageUrl`	A Business Central web-client deep link for one record.
Read	`whoAmI`	The calling principal, the company, and the endpoint posture.

PLANE	OPERATION	PURPOSE
Async	`status`	Track one async delivery request.
Async	`cancel`	Cancel one async delivery request.
Async	`requests`	List the caller's own async requests.
Read	`changes`	What the agent created, changed or deleted, and the actions and procedures it ran, with old and new values per field (chapter 15).
Execute	`invoke`	Run an enrolled wrapper codeunit by id and procedure name.
Execute	`action`	Run a named business action such as `post-sales` from the Action Catalog.
Write	`insert`	Create one record.
Write	`modify`	Update one record with optimistic concurrency.
Write	`delete`	Delete one record with optimistic concurrency.
Write	`batch`	Insert, modify and delete many records in one call and one transaction: all or nothing, with a preview (chapter 22).

The planes are not just documentation: the shipped permission sets grant Execute on exactly the handlers of one plane, so a principal holding `Wicket - Read` cannot run an Execute or Write operation at all (see chapter 8).

4. Security model in one page

- **Exposure list.** Table `Bcmag Exposure` holds one row per table or codeunit with `Allow Read / Insert / Modify / Delete / Invoke`. In the default mode, **Exposed tables only** (API name `Allowlisted`), a table without a row is invisible. `Open` mode (for sandboxes) allows any table the principal's BC permissions allow, but never `invoke` unless the separate `Allow Invoke In Open Mode` switch is set.
- **Read-Only Mode.** One switch on the setup card refuses every mutating operation (`insert`, `modify`, `delete`, `invoke`, `action`) before any other check. The exceptions are an action preview and a verb enrolled as non-mutating: both run inside a forced rollback that undoes their writes in Business Central (chapter 12 lists the limits). They can still **read and return** data: like every action they bypass the exposure list and the field policy, so on a read-only endpoint an enabled `document-pdf` serves any document's PDF and a posting preview returns amounts. Enable those verbs there only for principals who may see that, or mark them mutating to keep them off read-only.
- **Field policy.** Business Central has no column-level security, so Wicket decides per field whether it may be read or written, on every surface that names a field. Each field resolves to Hidden, Masked or Clear. Three modes: `AllFields`, `ByClassification` (the fresh-install default: `CustomerContent` and `SystemMetadata` are Clear, `EUII` and `EUPI` are Masked, everything else is Hidden until an administrator decides) and `Explicit` (a field is Hidden unless it has a row). A caller holding `Wicket - Unmask`, or a per-user exception, can see Masked fields in clear.
- **Catalog enrolment.** `invoke` runs only wrappers enrolled in the Procedure Catalog with `Allow Run`; `action` additionally needs an enabled Action Catalog row. There is no reflection over arbitrary AL methods.
- **Permission-set ladder.** `Wicket - Read` → `Wicket - Execute` → `Wicket - Edit` for callers, `Wicket - Audit` and `Wicket - Admin` for humans, and two generic data sets, `Wicket Data - Read` and `Wicket Data - Exec`. Each rung is the only grant of Execute on its handler codeunits and of write rights on one empty role-gate table (`Bcmag Execute Gate` from Execute up, `Bcmag Write Gate` from Edit up); the dispatcher checks the gate before routing, so the plane is enforced per principal even against a codeunit wildcard held elsewhere, and a refusal is a typed `403 role-forbidden` with no side effects.
- **Write hardening.** Writes re-fetch the record under `SecurityFilter::Filtered`, check the `@etag`, and roll back on failure. `modify` never changes a primary key (`400 pk-immutable`). Skipping table triggers (`init: false`, `validate: false`) is a tenant policy, off on fresh installs. Filters reach Business Central as parameterised, typed `SetFilter` values, never as concatenated strings.
- **Caller scoping.** Idempotency snapshots and reservations, async requests and their status are visible only to the principal that created them. Wicket's own audit tables can never be read or written through the data plane, in any mode.
- **Secrets.** The Azure storage credential lives in Isolated Storage as `SecretText`; the read SAS (a bearer fragment that is appended to package URLs, so it must be readable as text) lives there as non-debuggable text. Both are entered through a masked dialog

and never displayed again. A signed package URL is composed when a response is built and never stored: the request queue, its list page and the idempotency snapshot keep only the bare blob URL.

5. Prerequisites

REQUIREMENT	DETAIL
Business Central	Version 26 or later, SaaS. The core app targets platform 26.0 / runtime 16.0.
Dependencies	Core: Microsoft System Application 26.0. Companion: the core app and the Base Application.
Administrator access	Business Central: permission to install per-tenant extensions and to manage users and Microsoft Entra Applications. Azure: permission to create an app registration and grant admin consent.
For async delivery	An Azure Storage account and a private container.
For the MCP server	Nothing with the Claude Code plugin, on Windows (x64 or arm64) or an Apple-silicon Mac: the plugin downloads its server on the first start. Running from source, or for other MCP hosts: Bun 1.0 or later.
For building from source	VS Code with the AL Language extension, or the `alc` compiler, plus the BC 26 symbol packages.

6. Installing the extensions

The short path: the quick start puts this chapter, chapter 9's Simple Read-Only preset and the MCP install (chapter 34) on one page, for a first read-only connection to a sandbox.

From a compiled package

Download `Wicket_<version>.zip` (and, if you need posting actions, `Wicket_Actions_Base_<version>.zip`) from the repository's Releases page and unzip them; GitHub does not accept `.app` attachments, so each package ships zipped, with `SHA256SUMS.txt` to check them. Upload the core app in the Business Central admin center (**Manage Apps → Install Extension**) or through **Extension Management → Upload Extension**, and install it. Install the companion afterwards; it depends on the core app by id. The test app (`Wicket Tests`) is for development pipelines only and must not be installed in production.

From source

Each app lives in its own folder; never compile from the repository root, because the AL compiler globs `*.al` recursively and would swallow the sibling apps.

```bash
alc /project:./app     /packagecachepath:./.alpackages /out:Wicket.app
alc /project:./actions /packagecachepath:./actions/.alpackages /out:Wicket_Actions_Base.app
```

In VS Code open the `app/` folder (not the repository root), copy `app/.vscode/launch.json.example` to `launch.json` with your tenant and sandbox name, and press F5. The repository also carries an AL-Go pipeline that builds all three apps and runs the test suite in a BC 26 container on every push.

What installation does

- Creates the `Wicket Setup` singleton for every company, with fail-closed defaults: mode Exposed tables only, field policy `ByClassification` (CustomerContent and SystemMetadata Clear, EUII and EUPI Masked with the `PARTIAL` profile, every other classification Hidden), the **Unmask Tier Enabled** switch on, async off, Read-Only Mode off, **Require Key For Actions** on, **Allow Trigger Bypass** off.
- Leaves the exposure list empty. Nothing is reachable until you add rows, switch to `Open` mode, or apply a preset on the setup card (chapter 9).
- Arms the async maintenance task (the reaper) that fails stale workers, expires delivered packages and purges old queue rows. It re-arms itself every four hours, first thing in each run, so a fault later in the run cannot end the chain; its task failure codeunit (`Bcmag Async Reaper Guard`) re-arms it if even that did not stick, and every async submit re-arms it when no reaper task is live.
- Subscribes to company creation so a company created later gets its own setup row.
- The companion enrols its wrappers in the Procedure Catalog and its named actions in the Action Catalog, all with `Allow Run` off, and adds their exposure rows with `Allow Invoke` off. Re-installing or upgrading never overwrites an administrator's `Allow`

`Run` / `Allow Invoke` choices on those rows; an upgrade does refresh the companion-owned fields of rows that already exist (**Description** and **Wrapper Codeunit Id** everywhere, plus **Category**, **Is Mutating**, **Preview Supported** and **Args Schema** on Action Catalog rows), so documentation, routing and metadata fixes reach tenants that enrolled earlier.

Upgrading

Upgrades preserve every setting. Setup fields introduced after 1.0 are backfilled to their documented defaults; a tenant upgraded from 1.1 keeps field policy `AllFields`, so nothing that was readable becomes hidden. The two write-plane policies added in September 2026 likewise keep an upgraded tenant's old behaviour: **Require Key For Actions** starts off and **Allow Trigger Bypass** starts on (the internal marker field 202 makes this happen exactly once, so a later upgrade never overwrites an administrator's choice). Turn them to the fresh-install values when your callers are ready. The same upgrade stamps the storage destination onto queue rows written by earlier builds, once per company, so later changes to the Azure settings cannot misdirect their download URLs or cleanup. The product was renamed from BCMag to Wicket in October 2026 as a clean break: the permission sets carry new Role IDs (`Wicket - …`, `Wicket Data - Read`, `Wicket Data - Exec`, `Wicket Act`) and the pre-2026-09 aliases `Bcmag Rpc` / `Bcmag Rpc Admin` are gone, so re-assign permission sets after upgrading a tenant that ran a BCMag build.

7. Registering the caller

A caller is either a **service principal** (an application, the usual choice for agents and backends) or a **delegated user** (a person signing in interactively, used by consultants through the MCP server's device-code flow). Both need an Entra ID app registration; only the service principal needs a matching entry inside Business Central.

Azure side

1. **Microsoft Entra ID → App registrations → New registration.** Single tenant for internal use, multi-tenant if you will install across customer tenants. No redirect URI for service-to-service.
2. **Certificates & secrets → New client secret.** Copy the value at once. Prefer a certificate in production.
3. **API permissions → Add a permission → APIs my organization uses → Dynamics 365 Business Central.**
   - Service principal: **Application permissions →** **`app_access`**.
   - Delegated (device code): **Delegated permissions →** **`user_impersonation`**, and **Authentication → Allow public client flows → Yes**.
4. **Grant admin consent** for the tenant.
5. Note the Application (client) ID, the Directory (tenant) ID and the secret.

Business Central side (service principals only)

1. Search **Microsoft Entra Applications**, choose **New**.
2. Paste the client ID, give it a description, set **State** to `Enabled`.
3. On **User Permission Sets**, assign one Wicket role and one data set (chapter 8).
4. Save. Without this entry a valid token fails with `401 / "User does not exist"`; with the entry but no permission sets, every call returns 403.

A delegated user needs no Entra Applications entry. Their Business Central user account carries the permission sets instead.

Getting a token

```http
POST https://login.microsoftonline.com/<tenant-id>/oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id=<application-client-id>
&client_secret=<secret-value>
&scope=https://api.businesscentral.dynamics.com/.default
```

For a quick delegated token during testing:

```bash
az account get-access-token --resource https://api.businesscentral.dynamics.com --query accessToken -o tsv
```

The company GUID for the URL comes from the standard companies endpoint:

```http
GET https://api.businesscentral.dynamics.com/v2.0/<tenant-id>/<environment>/api/v2.0/companies
Authorization: Bearer <token>
```

8. Assigning permission sets

Wicket ships its permission sets as composable AL objects. Assign exactly **one role** for the endpoint and **one or more data sets** for the business tables in scope. A role grants access to Wicket's own objects and to the handlers of its plane; it grants nothing on business data.

The roles

PERMISSION SET	WHO GETS IT	WHAT IT ALLOWS
`Wicket - Read`	A read-only agent	`discover`, `describe`, `query` (including aggregates and async delivery), `readBlob`, `capabilities`, `skill`, `pageUrl`, `whoAmI`, and its own `status` / `requests` / `cancel`. Writes only its own audit rows and queue rows.
`Wicket - Execute`	An agent that may run enrolled business logic	Everything in Read plus `invoke` and `action`. The generic write plane stays closed.
`Wicket - Edit`	A full read/write agent	Everything in Execute plus `insert`, `modify`, `delete`, `batch`.
`Wicket - Audit`	Support, security review	Opens every Wicket admin page and reads every Wicket table, including the logs, the request queue and the change history (it includes `Wicket - Audit Changes`). Cannot change anything, purge logs or enter credentials.
`Wicket - Audit Changes`	The customer's administrator, who reviews what the agent changed	Opens **Wicket Changes** and reads every user's entries; through the `changes` operation, a caller holding it sees every user's changes and may filter by `user`. Included by `Wicket - Audit` and so by `Wicket - Admin`. Role ID `WICKET - CHANGES`.
`Wicket - Admin`	The operator	The whole control plane: setup, exposure, field policy, catalogs, credential entry, log purge. Includes Edit and Audit.
`Wicket - Unmask`	A principal that must see masked fields in clear	A data grant, not a rung: lifts every Masked verdict to Clear (never Hidden), while Setup **Unmask Tier Enabled** is on. Not included by `Wicket - Admin`; assign it next to Read, Execute or Edit. `SUPER` holds it implicitly.
`Wicket Data - Read`	A read-only agent that may read any table	Includes `Wicket - Read` and adds Read on every table in the tenant plus Execute on all tables, pages and queries.
`Wicket Data - Exec`	An agent that may read anything and run business logic	Includes `Wicket - Execute` and `Wicket Data - Read` and adds Execute on every codeunit, report and XMLport. Direct table data stays read-only, and `insert` / `modify` / `delete` stay refused (`403 role-forbidden`): the codeunit wildcard reaches the write handlers, but the Write Gate is granted only by `Wicket - Edit`.
`Wicket Act` (companion)	Any principal that may run the companion's actions	Execute on the companion's wrappers and the Base Application routines they call (posting, preview, release, quote conversion, availability), Read on the documents and setup they inspect. Combine with `Wicket - Execute` or above and the standard Business Central permission sets for the work itself (posting, document editing, reports).

Recipes

NEED	ASSIGN
Read-only agent over everything	`Wicket Data - Read`, and the **Simple Read-Only** preset on the setup card (chapter 9)
Read-only agent over a few named tables	`Wicket - Read` plus a customer set with Read on those tables
Agent that reads everything and may post documents	`Wicket Data - Read`, `Wicket - Execute`, `Wicket Act`, and the standard sales/purchase posting sets the wrappers rely on
Agent that must see masked fields in clear	`Wicket - Read` (or higher) plus `Wicket - Unmask`; for one field or a limited time use an exception row instead
Agent with the generic write plane on named tables	`Wicket - Edit` plus a customer set with the needed Insert/Modify/Delete bits
Operator who configures Wicket	`Wicket - Admin`
Support or security reviewer	`Wicket - Audit`
Job Queue user, if you schedule the log purge (codeunit 94121) yourself	`Wicket - Admin` (the automatic purge needs no user)

What to know before assigning the wildcard sets

- `Wicket Data - Read` uses `tabledata * = R`. The wildcard spans every extension and every system table in the tenant; that is what "read any information" means. Inside Wicket the exposure list and the field policy still apply, so the wildcard only supplies the Business Central side of the check.
- `Wicket Data - Exec` grants Execute on every codeunit, Wicket's own write handlers included: a permission set cannot carve an exception out of a wildcard. The generic write plane nevertheless stays closed to it because the dispatcher routes `insert` / `modify` / `delete` only for principals that can write the `Bcmag Write Gate` table, which only `Wicket - Edit` grants. Separately, an object that carries its own `Permissions` property can write while it runs, which is how posting routines create ledger entries for users who hold no ledger permission. The set therefore means "cannot write directly", not "cannot cause a write". For production prefer `Wicket Data - Read` + `Wicket - Execute` + the provider's own wrapper set, which names exactly the codeunits in scope.
- Permission sets are additive. No Wicket set restricts what a principal holds elsewhere; the role gates restrict only what Wicket itself will route.
- A principal whose role is below an operation is refused before the handler runs with a `403 role-forbidden` problem whose detail names the permission set required. Do not retry; assign that rung.
- `SUPER` cannot be assigned to an application identity. For sandbox testing `D365 BUS PREMIUM` is an acceptable data set.

9. The Wicket Setup card

Search **Wicket Setup**. The card holds one row per company. Fields are grouped as on the page.

Presets

The quickest way from install to a working endpoint is one of the four **Presets** on the card's action bar. Each asks for confirmation, sets only the switches listed below, and then names the permission sets to assign to the caller (chapter 7, Business Central side). A preset never assigns permissions itself: granting access stays in the standard **Users** and **Microsoft Entra Applications** pages. The **Current Preset** field at the top of the card shows which preset the settings match, or `Custom` (a fresh install is `Custom`: Exposed tables only, and writable).

PRESET	MODE	READ-ONLY MODE	ALLOW INVOKE IN OPEN MODE	FIELD POLICY MODE	ASSIGN THE CALLER
Simple Read-Only	Open	on	off	unchanged	`Wicket Data - Read`
Read-Only (Selected Tables)	Exposed tables only	on	off	unchanged	`Wicket - Read` plus a set with Read on the allowed tables. The Exposure list opens next so you can allow tables.
Read + Business Actions	Open	off	off	unchanged	`Wicket Data - Read`, `Wicket - Execute`, `Wicket Act`
Developer Sandbox	Open	off	on	AllFields	`Wicket - Admin`. Refused in a production environment.

Every preset except Developer Sandbox leaves the field policy and masking as they are, so hidden and masked fields stay hidden and masked; opening tables is a separate decision from opening columns. The audit, queue, access-exception and field-exposure tables stay unreadable in every mode. Under Read + Business Actions, `insert`, `modify` and `delete` are still refused (`403 role-forbidden`) for any caller below `Wicket - Edit`, and `invoke` stays deny-by-default; enrolled actions are gated by the Action Catalog as usual. Both read-only presets also switch Allow Invoke In Open Mode off, so turning Read-Only Mode off later does not quietly open `invoke`.

General

FIELD	DEFAULT	MEANING
Read-Only Mode	off	Refuses `insert`, `modify`, `delete`, `invoke` and mutating `action` calls for every caller, before any other check. Action previews and verbs enrolled non-mutating still run, inside a forced rollback (chapter 12); they can return data, so mark such a verb mutating to keep it off a read-only endpoint. Turn it on for a tenant that must never be written to through Wicket.
Mode	Exposed tables only	**Exposed tables only** (API name `Allowlisted`) honours the exposure table. `Open` allows read and write on any table the principal's BC permissions allow; use it in sandboxes only.

FIELD	DEFAULT	MEANING
Allow Invoke In Open Mode	off	Even in `Open` mode `invoke` stays deny-by-default unless this is on.
Default Languages	`en-US`	Comma-separated culture tags used for `captionML` when a request passes no `languages`.
Skill Doc Url	blank	Public HTTPS URL of the usage guide served by the `skill` operation. Blank disables the operation. HTTP is refused.

Limits

FIELD	DEFAULT	MEANING
Default Top	100	Rows returned when a query passes no `top`.
Max Top	1000	Hard cap on `top`.
Per-Call Scan Budget	100 000	Rows a single synchronous query page may examine before it fails with `422 scan-budget`. Keyset paging (no `orderBy`) counts only the page's own rows; offset paging (with `orderBy`) also counts the rows it skips. Async delivery is not bound by it.
Max Filter Length	2048	Characters; longer filters fail with `413`.
Max Select Fields	256	Fields per `select`; more fail with `413`.
Decimal String Threshold	2^53	Decimals with a larger magnitude cross the wire as strings to protect precision.

Writes and log

FIELD	DEFAULT	MEANING
Require ETag On Writes	off	When on, a `modify` or `delete` without `data.etag` fails with `409 etag-required` instead of being last-write-wins.
Strong ETag	off	Folds an HMAC-SHA256 of the row's values, keyed with a per-company tenant key, into `@etag`, closing the same-millisecond window. The key means hidden and masked values cannot be brute-forced offline from a token. Costs a field read and hash per row. Flipping it invalidates every outstanding token.
Require Key For Actions	on (fresh install), off (upgraded)	`invoke` and `action` calls without an envelope `idempotencyKey` are refused with `400 idempotency-key-required`. A wrapper may post and commit; only a key lets a caller retry a lost response without posting twice. An action preview and a non-mutating verb need no key: they run inside a forced rollback and cannot take effect.
Allow Trigger Bypass	off (fresh install), on (upgraded)	Whether `insert` may pass `init: false` (skip the table's OnInsert trigger) and `modify` may pass `validate: false` (skip field OnValidate). Off, such calls are refused with `403 trigger-bypass-disabled`.
Operation Log Retention Days	30	The purge job deletes Operation Log rows, idempotency snapshots and settled idempotency reservations older than this; the reaper applies the same retention to finished async requests (chapter 15). 0 disables purging.
Log Request Details	on	Records each request's shape in the Operation Log's **Request Details**: `select`, `filter`, `orderBy`, `top`, the change window, the `aggregate` spec, and for a write or an invoke the **names** of the fields or arguments it carried, never their values. A filter can contain a value the caller searched for (a name, an e-mail), so a tenant that must not store that turns this off. The table and target are recorded either way. Switched on once by the 2026-10 upgrade on existing tenants.

Field policy

FIELD	DEFAULT	MEANING
Field Policy Mode	ByClassification (fresh install), AllFields (upgraded from 1.1)	See chapter 11. Per-classification verdicts live on **Wicket Classification Rules**, not on Setup.
Unmask Tier Enabled	on (off on a tenant upgraded from a version without it)	Whether the `Wicket - Unmask` permission set lifts Masked fields to Clear for its holders. `SUPER` holds it implicitly, so turn this off to keep `SUPER` from unmasking. Chapter 11.
Describe Restricted Fields	off	When on, `describe` lists hidden fields with `readable: false` instead of omitting them. Masked fields are always listed (with `access: "masked"`).

Change History

FIELD	DEFAULT	MEANING
Record Changes	Old and New Values (fresh install and upgrade)	`Off`: no change history. `Fields Only`: which fields changed, never their values. `Old and New Values`: each changed field's old and new value, as the field policy shows the field (a masked field keeps its mask, a hidden one only that it changed). A change applies from that moment; switching to a lower level does not rewrite what was recorded.
Change Retention Days	90	Entries older than this are deleted by the maintenance task (the reaper), on its own schedule, independent of the Operation Log retention. 1 to 3650.

FIELD	DEFAULT	MEANING
Max Value Length	250	Longer values are cut, and the line is marked Truncated. 20 to 2048.
Agent Can Read Changes	on	Whether the agent may read the history through the `changes` operation (`wicket_changes`). Off keeps it for people only, on the **Wicket Changes** page.

API Routing

FIELD	DEFAULT	MEANING
API Routing	Off	`Prefer` lets a client (the MCP server's `wicket_api_read`) read through one of the tenant's own API pages when one covers the data, instead of Wicket's dynamic plane. Before each such read the client asks `describe kind:"api"` , which answers `routable` only when this switch is `Prefer` and Wicket would let the caller read the API's source table, and marks each API field `readable` under the field policy. The API read itself does not pass the endpoint, so it is not recorded in the Operation Log. `Off` : clients read through `query` only. The API catalog (`discover kind:"apis"`) is visible either way.

Async Delivery (Azure Blob)

FIELD	DEFAULT	MEANING
Async Enabled	off	Master switch. Off, a `deliver` block is refused with `403 async-not-configured` .
Azure Storage Account	blank	Account name.
Azure Blob Container	blank	Private container that receives the packages.
Azure Auth Mode	Shared Key	`Shared Key` signs each request with the account key; `Ready SAS` uses a pre-provisioned SAS token. The secret itself is entered through **Set Azure Credential**.
Download Mode	Service SAS	How the returned package URL grants access: `Service SAS` (Wicket signs a per-blob, read-only, https-only SAS; requires Azure Auth Mode `Shared Key`) or `Ready SAS` (appends the token entered through **Set Azure Read SAS**). A stored Ready SAS must grant read and nothing else (`sp=r` : every package holder receives the token, so list would let them enumerate and download every other caller's package, and write or delete would let them alter or remove packages), and must not expire before a package delivered now would (Package TTL Hours); otherwise requests are refused until a valid token is stored. `Stream Proxy` is reserved and refuses every request while selected. An incomplete combination refuses new requests with `403 async-not-configured` and the worker never reports a package delivered without a usable download configuration.
Default Package Format	NDJSON	`NDJSON` , `JSON` or `ZIP` when the request names none.
Package TTL Hours	24	Retention of a delivered package, counted from the moment the request **finished** (`expiresAt` = `completedAt` + TTL, so a scheduled or long-running job keeps the full TTL). After it the reaper marks the request `expired` and deletes the blob. Deletion is tracked separately (**Blob Cleanup** on the request queue): a delete that cannot be confirmed stays `Pending` and is retried on later reaper runs, up to a cap, for expired and cancelled requests alike; a row still pending at the cap is the operator's signal that storage needs attention (telemetry `WICKET-0002`). The delete addresses the account, container and endpoint recorded on the request when its package was uploaded, not the current setup values: repointing Azure Storage Account or Azure Blob Container afterwards neither redirects cleanup to the wrong place nor lets it confirm absence there; a package in a location the current credential can no longer open stays `Pending` .
SAS TTL Minutes	60	Lifetime of a Service SAS download URL. `status` mints a fresh one on every call (valid from 15 minutes back, to allow for clock skew, until now + this TTL, but never past the package's `expiresAt`), so re-read `status` for a working link rather than storing one.
Max Schedule Ahead Days	30	Furthest future `scheduleAt` accepted.
Max Active Requests	20	Non-terminal requests one caller may hold; more return `429` .
Azure Blob Endpoint	blank	Override of the blob host for sovereign clouds. Blank composes `https://<account>.blob.core.windows.net` .

Two actions on the card, **Set Azure Credential** and **Set Azure Read SAS**, open a masked dialog and store the value in Isolated Storage. The value is never shown again; re-run the action to replace it.

Recommended production posture

Read-Only Mode on unless writes are a requirement; API Routing `Off` unless the use case wants API-first reads and accepts that those reads are audited only by the client; mode Exposed tables only; field policy `ByClassification` with only the classifications the use case needs Clear, and `Wicket - Unmask` assigned only where a human or agent must see masked values; Require ETag On Writes on for any writing agent; Strong ETag on when several agents write the same tables; Require Key For Actions on and Allow Trigger Bypass off (the fresh-install values; set them on an

upgraded tenant once its callers send keys and run triggers); retention set to your audit requirement; async enabled only with `Service SAS` and a dedicated container.

10. Exposing tables and codeunits

Search **Wicket Exposure**. One row per object.

FIELD	MEANING
Object Type	`Table` or `Codeunit`.
Object Id	The object number, for example `18` for Customer.
Allow Read	`discover`, `describe`, `query`, `readBlob`, `pageUrl`. On by default for a new row.
Allow Insert / Allow Modify / Allow Delete	The corresponding write operation.
Allow Invoke	Codeunit rows only: the wrapper may be run through `invoke`.
Description	Free text for your own bookkeeping.

Rules the table enforces regardless of what you enter:

- Wicket's own tables (94100–94199) can never be written through the data plane, in any mode. The Operation Log, the Idempotency Log, the Idempotency Reservation and the Request Queue can never be read through it either, because they hold every principal's rows; neither can the Access Exceptions (each user's grants and reasons) or the Field Exposure rows (which would name every field an administrator hid, while `describe` keeps a hidden field indistinguishable from a missing one). `discover` does not list them, in `Open` mode included. Setup, Exposure, the catalogs, the Classification Rules and the Mask Profiles remain readable as fixtures; `capabilities` publishes the rules and profiles anyway.
- A row grants nothing the principal's Business Central permissions do not also grant. `describe` reports the effective `permissions` per table.
- `discover` lists only tables that are exposed and readable by the caller, so an agent sees exactly the surface you opened.

A typical exposure list for read-only reporting:

OBJECT TYPE	OBJECT ID	ALLOW READ	DESCRIPTION
Table	18	on	Customer
Table	27	on	Item
Table	36	on	Sales Header
Table	37	on	Sales Line
Table	21	on	Cust. Ledger Entry

11. Field-level policy

Business Central has no per-column security. Wicket adds it in its own projection layer and applies it wherever a field is named: `select`, `selectFieldNo`, `select ["*"]`, `filter`, `orderBy`, `aggregate`, `readBlob`, `insert`, `modify` and `describe`.

Verdicts

Every field resolves, per caller, to one of three read verdicts.

VERDICT	WHAT THE CALLER GETS
Hidden	The field does not exist for the caller. Naming it answers `403 field-restricted`; `select ["*"]` skips it; `describe` omits it.
Masked	The field is returned as a mask rendering (for example `****45`), and the row names it in `"@masked": [field names]`. It can be read but never filtered, sorted, aggregated or written back.
Clear	The stored value.

Writing is separate: a field is writable only through its own row's **Allow Write** (or, in `ByClassification` mode without a row, when its classification's baseline verdict is Clear). Exceptions and the Unmask tier never change writability, and a Masked field can be writable without ever being readable in clear.

Modes

MODE	BEHAVIOUR	WHEN TO USE
AllFields	Every field Clear and writable; rules and rows ignored.	Back-compat for tenants upgraded from 1.1; sandboxes.
ByClassification	A field's verdict comes from its per-field row if it has one, otherwise from the classification rule for its `DataClassification`.	Production default.
Explicit	A field without a per-field row is Hidden. Rows are the only way a field becomes visible.	Maximum lockdown; a curated field surface per table.

Primary-key fields are always Clear and writable; they address rows.

Classification rules

Wicket Classification Rules holds one row per data classification: Read Access (Hidden, Masked, Clear) and, for Masked, a Mask Profile. Install seeds the fresh-install posture; a missing row falls back to the same built-in value, so deleting a row restores the default rather than opening or closing a class by accident. Upgrades add only missing rows and never overwrite a choice.

CLASSIFICATION	FRESH-INSTALL VERDICT
CustomerContent	Clear
SystemMetadata	Clear
EndUserIdentifiableInformation (EUII)	Masked, profile `PARTIAL`
EndUserPseudonymousIdentifiers (EUPI)	Masked, profile `PARTIAL`
ToBeClassified, OrganizationIdentifiableInformation, AccountData	Hidden

The classification comes from the field's own `DataClassification` in the object's metadata, plus the Wicket rows below. Business Central's per-company Data Sensitivity worksheet is not a source: it is OnPrem-scoped (the table and the page), so a SaaS extension can neither read it nor open it. A field the app author left unclassified is `ToBeClassified` and Hidden.

Check what the default actually masks. Business Central classifies most customer and vendor contact data (name, address, phone, e-mail on Customer and Vendor) as `CustomerContent`, which is Clear by default; the EUII/EUPI default mainly catches user identifiers. Before relying on the default, run `describe` on the tables an agent will read and look at each field's `dataClassification`, then add Masked rows for the fields you want masked. Mask derived fields together with their source: Customer `Search Name` is copied from `Name`, so masking `Name` alone leaves the same value Clear in `Search Name`.

Per-field rows

Wicket Field Exposure rows override the classification rule for one field, and in `Explicit` mode they are the only thing that makes a field visible.

FIELD	MEANING
Table Id, Field No.	The field.
Read Access	Hidden, Masked or Clear. Clear by default for a new row, because a row is usually a grant.
Mask Profile	Used when Read Access is Masked; blank means the built-in partial mask.
Allow Write	Off by default.
Description	Your note.

Precedence

1. `AllFields` mode: Clear. A primary-key field: Clear.

2. The baseline is the field's row if it has one; otherwise Hidden in `Explicit` mode; otherwise the classification rule.
3. A per-user exception (below) active now replaces the verdict and the profile. A field-scope exception wins over a classification-scope one.
4. A caller who holds the Unmask tier has a Masked verdict lifted to Clear, unless a per-user exception decided the field in step 3: an exception is final, so a row that masks a field for one user stays masked even for a holder of the tier. Hidden is never lifted.
5. A request carrying `"unmask": false` takes the more restrictive of the baseline and the result of steps 3 and 4 (Hidden, then Masked, then Clear), so a caller can always opt out of its own grants.

Mask profiles

Wicket Mask Profiles define how a Masked value is rendered. Profiles are referenced by rules, field rows and exceptions. A blank or unknown profile code renders as the built-in partial mask, so a deleted profile can never fall open to Clear. Install seeds five.

KIND	EXAMPLE
Keep Last (`PARTIAL` : last 2)	`1503862199` becomes `****99` . The hidden run is a fixed four characters unless Preserve Length is on.
Keep First / Keep First and Last	Keep N leading and/or trailing characters, mask the rest.
Email (`EMAIL`)	`jon.smith@example.com` becomes `j****@example.com` : the first letter and the domain. When the rest of the local part is shorter than Min Hidden, the whole local part is masked (`jo@x.com` becomes `****@x.com`).
Fixed Text (`REDACT`)	`[redacted]` (the Fixed Text member; a blank Fixed Text renders `[redacted]` too).
Hash (`HASH`)	`h:` plus 16 hex characters. Equal values give equal tokens, so an agent can see that two rows share a value without seeing it.
Null (`NULL`)	`null` .

Two members protect against leaking by shape. **Min Hidden** (default 4) is the minimum number of characters a rendering must hide: a value too short to keep its ends and still hide that many is masked completely. **Preserve Length** (default off) makes the hidden run as long as the value; with it off the run is a fixed four characters, because the length of a value is itself information.

Non-text fields (numbers, dates, GUIDs and the like): `Hash` and `Fixed Text` still render, as strings, and `describe` reports the field's emitted type as a string. The exception is `Hash` on a Boolean or Option/Enum field, which renders `null` and keeps the field's own type (see below). Every other kind renders `null` . Binary fields are never rendered; see the refusals below.

The `Hash` kind, and the strong ETag, use a keyed hash (HMAC-SHA256) under a per-company key held in the tenant's isolated storage. An unkeyed hash of a short hidden value could be brute-forced offline from the token alone; the key never leaves the tenant. Rotating the key is not exposed in this version; it is created per company at install or upgrade. A hash is pseudonymisation, not anonymisation: the token still identifies a person to anyone who can compare it with another token from the same tenant, so it remains personal data. On Boolean and Option/Enum fields `Hash` renders `null` (and `describe` reports kind `null`): with only a handful of possible values, knowing one row's value would reveal every row with the same token; such fields should be Hidden or Clear. Do not grant **Allow Write** on a `Hash` -masked field: a caller who can write `v` and read back its token has a hash oracle, and for low-entropy values (a national ID, a PIN, a short code) can recover any other row's value by trying candidates.

Exceptions

Wicket Access Exceptions gives one user a different verdict than the baseline, optionally for a time window.

FIELD	MEANING
User Security ID	The user or application the row applies to.
Scope	`Field` (one table field) or `Classification` (every field of one data classification).
Read Access, Mask Profile	The verdict the user gets instead. An exception can loosen (Masked to Clear) or tighten (Clear to Hidden).
Valid From, Valid Until	Blank means open-ended. An expired row has no effect.

FIELD	MEANING
Reason	Why the grant exists.

Example: let one support user read customer e-mail in clear for a week.

SCOPE	TABLE ID	FIELD NO.	READ ACCESS	VALID UNTIL	REASON
Field	18	102	Clear	2026-10-11	Ticket 4411

The `Wicket - Unmask` permission set is the coarse alternative: it lifts every Masked verdict to Clear for the principal that holds it, for all classifications. It never lifts Hidden, and a per-user exception that matches the field (field- or classification-scope) still decides it: an exception row that masks a field for a user keeps it masked even when that user holds the tier. It is its own set, not part of `Wicket - Admin`: administering the policy and reading the data it protects are separate decisions. Assign it next to a data-plane rung (Read, Execute or Edit). The tabledata wildcard sets do not open it. `SUPER` holds every table permission and so holds it implicitly; the Setup switch **Unmask Tier Enabled** (default on; off on a tenant upgraded from a version that predates it) turns the tier off for everyone, `SUPER` included. For a grant narrower than "every classification", use exception rows.

Opting out. A request can carry `"unmask": false` in `data` to receive the baseline view, as if it held no exceptions and no Unmask tier. The MCP server stamps this on every read operation of a connection whose settings have `unmask: false`, so an agent connection can be kept at the masked view even when its principal is privileged.

Audit. The Operation Log column **Unmasked Fields** lists the `table:field` pairs a request read in clear although the baseline masks them (lifted by an exception or the Unmask tier); `,+N` means N more were left out. It is written for value-returning operations only (`query`, `readBlob`). A lift that only affected `describe`, `select ["*"]` or a suggestion reveals no value and is not logged.

What the caller sees and what is refused

A masked field is never usable as a lever on the hidden value.

REQUEST	ANSWER
`filter`, `orderBy` or `aggregate` over a masked field	`403 field-masked`. A filter would let a caller recover the value by binary search, an ordering or `min` / `max` leaks it directly.
An effective key, or a `modifiedAfter` / `modifiedBefore` window, that would reconstruct a masked value	`403 field-masked`.
`readBlob` of a masked binary field	`403 field-masked`; bytes have no partial form.
Replay (chapter 18) of a stored response that carries a field now masked, or was filtered, ordered or aggregated over one	`403 replay-withheld` with reason `field-masked`.
`modify` that sends a masked field's own rendering back (a client echoing `****45`)	`400 masked-value`: the stored value would be overwritten with the mask. A blank value is not judged. Send only the fields you change. An `insert` is not judged, because there is no stored value to compare with: an insert copied from a masked read stores the mask as data, so never build an insert from a masked row.

`describe` gives each readable field `access: "clear"` or `"masked"`. A masked field also carries `mask: { profile, kind }`, `filterable: false` and `sortable: false`, and its `type` is the emitted type. `capabilities.fieldPolicy.maskProfiles` reports each profile's configured kind (for example `hash`); `describe` reports the effective kind per field, which is `null` for a Boolean or Option/Enum field under a `Hash` profile. Each row of a `query` that carries masked members names them in `"@masked"`.

What masking does not protect

Masking is minimisation for agents, not data-loss prevention and not Business Central data security. Read this before describing a field as "protected".

- It governs the Wicket plane only. Anyone with direct API, OData or web-client access to the table sees clear values.
- `invoke` and `action` return values carry no field identity, so they are not masked. This was already true for hidden fields.
- A FlowField is judged by its own classification, not by the fields it calculates from.
- Primary-key fields are always Clear, and `@recordId` embeds the primary-key values in cleartext. A table keyed by a sensitive value is not protected.

- Reads routed through a tenant API page (`wicket_api_read`) are not served by Wicket and cannot be masked; a field Wicket masks is treated as not readable for routing, so such a read falls back to the dynamic plane.
- Audit gaps: the **Unmasked Fields** column does not see everything a grant discloses.
  - A read routed through a tenant API page (`wicket_api_read`) that a lift made possible is not logged: `describe` kind `"api"` answers `readable: true` because of the lift, the client then reads clear values from the API page, and neither call writes a lift.
  - An exception that loosens the profile but keeps the field Masked (for example `HASH` to a keep-last profile) is not counted as a lift, although it shows more of the value.
  - Unmasks performed by the async delivery worker are not written to the Operation Log.
  - Idempotency snapshots of a lifted read hold the clear values, and holders of `Wicket - Audit` can read them.
- Residuals: a stored response replays under a profile that has since been tightened, until its idempotency window ends (replay does refuse a response over a field that has become masked or hidden); and if a custom permission set grants a principal no Read on **Wicket Access Exception**, tightening exceptions are not applied for that principal.
- Packages already delivered to Azure storage are not masked retroactively when the policy tightens.

12. Enrolling procedures and actions

Procedure Catalog (raw `invoke`)

Search **Wicket Procedure Catalog**. A procedure is callable only when a row exists with `Allow Run` on **and** the wrapper codeunit has an exposure row with `Allow Invoke` on.

FIELD	MEANING
Logical Codeunit Id, Procedure Name	What the caller addresses: `data.codeunitId` and `data.procedure` .
Wrapper Codeunit Id	The small, auditable codeunit that actually runs. Usually equal to the logical id.
Args Schema	Optional JSON schema surfaced by `describe` .
Return Shape	`json` (the wrapper returns an object) or `void` (the caller gets `{ ok: true }`).
Allow Run	The switch. Off by default.
Description	Surfaced by `describe` .

The core ships one sample wrapper, `Bcmag Wrap Ping` (codeunit 94190, procedure `Ping`). It is not enrolled; enrol it yourself to verify the invoke plane end to end.

Action Catalog (named `action`)

Search **Wicket Action Catalog**. An action is a stable verb over a wrapper.

FIELD	MEANING
Action Code	The verb, for example `post-sales` . Stored in upper case (a Code field); `capabilities.actions` lists it in lower case and calls match it in any case.
Description, Category	Surfaced through `capabilities.actions` .
Wrapper Codeunit Id	Any invoke wrapper.
Is Mutating	Enforced. Off, the wrapper runs inside a **forced rollback**: explicit commits are ignored, a nested `Codeunit.Run` that would commit is refused, and every write is undone when it returns, while its result still reaches the caller. Such a verb can run on a read-only endpoint and needs no idempotency key. Not undone by any rollback: what the wrapper sends outside Business Central (HTTP calls, background sessions it starts, number sequences). An upgrade of the companion never turns Is Mutating off on an existing row, so an administrator who marks a verb mutating keeps it so.
Preview Supported	Whether `preview: true` is accepted. A preview runs inside the same forced rollback, but it cannot take the nested-run guard (Business Central's posting preview is itself such a run), so it relies on the wrapper honouring `args.preview` . The companion's four posting verbs do, and are the only verbs enrolled with it; do not switch it on for a wrapper that ignores `args.preview` .
Allow Run	The switch. Off by default.

FIELD	MEANING
Args Schema	Optional JSON schema of the arguments, surfaced as `argsSchema` in `capabilities.actions` when it parses as a JSON object. The companion fills it for every verb.

`capabilities.actions` lists **every** row, enabled or not, with `enabled` = Allow Run, so an agent can tell "no companion" (empty) from "enrolled but disabled" and name the administrator step.

Enabling an action grants whatever its wrapper does, independent of the exposure list and the field policy. Review a wrapper like a permission set before enabling it.

The companion's actions

ACTION	PROCEDURE (FOR `INVOKE`)	WRAPPER	ARGUMENTS
`post-sales`	`PostSalesDocument`	94200	`systemId` or `documentType` + `no`; `ship`, `invoice`, `receive` (defaults ship + invoice; a return order defaults to receive + invoice)
`post-purchase`	`PostPurchaseDocument`	94202	`systemId` or `documentType` + `no`; `receive`, `invoice`, `ship` (defaults receive + invoice; a return order defaults to ship + invoice)
`post-gen-journal`	`PostGeneralJournal`	94203	`template`, `batch`
`post-item-journal`	`PostItemJournal`	94204	`template`, `batch`
`release-sales`	`ReleaseSalesDocument`	94211	`systemId` or `documentType` + `no`
`reopen-sales`	`ReopenSalesDocument`	94212	`systemId` or `documentType` + `no`
`release-purchase`	`ReleasePurchaseDocument`	94213	`systemId` or `documentType` + `no`
`reopen-purchase`	`ReopenPurchaseDocument`	94214	`systemId` or `documentType` + `no`
`make-order-from-quote`	`MakeOrderFromQuote`	94215	`systemId` or `no` of a sales quote
`make-invoice-from-quote`	`MakeInvoiceFromQuote`	94216	`systemId` or `no` of a sales quote
`make-purchase-order-from-quote`	`MakePurchaseOrderFromQuote`	94217	`systemId` or `no` of a purchase quote
`document-pdf`	`DocumentPdf`	94218	`document` (`salesQuote`, `salesOrder`, `salesInvoice`, `postedSalesInvoice`, `postedSalesCreditMemo`, `postedSalesShipment`, `purchaseQuote`, `purchaseOrder`, `postedPurchaseInvoice`, `postedPurchaseCreditMemo`); `systemId` or `no`
`item-availability`	`ItemAvailability`	94219	`itemNo`; `locationCode`, `variantCode`, `date` (default the work date), `quantity`

The four posting verbs accept `preview: true` (chapter 23). `document-pdf` and `item-availability` are enrolled with **Is Mutating** off: they run inside the forced rollback, so a report that stamps *No. Printed* leaves no trace, they work on a read-only endpoint and they need no idempotency key. The quote conversions take the same path as **Make Order** / **Make Invoice** on the quote page: the customer or vendor is checked, the lines are copied, the quote is archived when the setup says so and then deleted. A sales quote tied to an opportunity in progress is refused, because Business Central would open the *Close Opportunity* page there. Release and reopen take the same path as the Release and Reopen actions on the document page, so the approval workflow and prepayment rules apply: a document pending approval can be neither released nor reopened, and a sales or purchase order with an unpaid prepayment becomes *Pending Prepayment* instead of released.

The journal wrappers call the batch-posting codeunits directly (the interactive front ends would ask questions or, with *Post with Job Queue*, only schedule the post), but they apply the same business refusals those front ends apply first: a template with **Force Posting Report** must be posted through its report, and a Jobs general-journal template whose Source Code is Source Code Setup's **Job G/L WIP** code is refused (those lines belong to the WIP process). Results are described in chapter 23.

To enable posting of sales documents: set `Allow Run` on the `post-sales` row of the Action Catalog (for `action`) and, if raw `invoke` is also wanted, `Allow Run` on the Procedure Catalog row for 94200 plus `Allow Invoke` on the exposure row for codeunit 94200. Revoke by switching the flag off; deleting a row is not durable because re-enrolment re-inserts it disabled.

Adding your own action

Any extension can add verbs to the Action Catalog without a change to Wicket: the companion is one such provider, not a special case. A provider needs three things: a **wrapper codeunit** that does the work, an **enrolment** that writes the catalog row when

the app is installed or upgraded, and a **permission set** for the principals that may run it. The example below adds `block-customer`, which sets or clears a customer's *Blocked* field. It compiles against Wicket 1.3.0.0.

1. Depend on Wicket. In the provider's `app.json` (the Base Application comes with `application`, which `Customer` needs):

```json
"dependencies": [
  { "id": "4057cf1f-1b34-409a-a570-c6fdf625c434", "name": "Wicket", "publisher": "Dave The Wicketkeeper", "version": "1.3.0.0" }
]
```

2. Write the wrapper. Its `OnRun` reads the call's arguments from the single-instance `Bcmag Invoke Context`, does the work, and hands an object back with `SetResult`. `target.systemId` from the call arrives in the arguments as `systemId`. An `Error` raised here reaches the caller as `422 action-failed` with the error text in `detail`.

```al
codeunit 50100 "Contoso Act Block Customer"
{
    // Action wrapper for `block-customer`: set or clear a customer's Blocked field.
    // args:   { systemId | no, blocked: "none" | "ship" | "invoice" | "all" }
    // result: { no, blocked, changed }

    trigger OnRun()
    var
        Customer: Record Customer;
        Ctx: Codeunit "Bcmag Invoke Context";
        Args, Result : JsonObject;
        Tok: JsonToken;
        Id: Guid;
        Before, NewBlocked : Enum "Customer Blocked";
        BlockedName: Text;
    begin
        Args := Ctx.GetArgs();

        if Args.Get('systemId', Tok) then begin
            if not Evaluate(Id, Tok.AsValue().AsText()) then
                Error('systemId is not a GUID.');
            if not Customer.GetBySystemId(Id) then
                Error('No customer has systemId %1.', Id);
        end else
            if Args.Get('no', Tok) then
                Customer.Get(Tok.AsValue().AsText())
            else
                Error('Send systemId or no.');

        if not Args.Get('blocked', Tok) then
            Error('blocked is required: none, ship, invoice or all.');
        BlockedName := LowerCase(Tok.AsValue().AsText());
        case BlockedName of
            'none':
                NewBlocked := NewBlocked::" ";
            'ship':
                NewBlocked := NewBlocked::Ship;
            'invoice':
                NewBlocked := NewBlocked::Invoice;
            'all':
                NewBlocked := NewBlocked::All;
            else
                Error('blocked must be none, ship, invoice or all.');
        end;

        Before := Customer.Blocked;
        Customer.Validate(Blocked, NewBlocked);
        Customer.Modify(true);

        Result.Add('no', Customer."No.");
        Result.Add('blocked', BlockedName);
        Result.Add('changed', Customer.Blocked <> Before);
        Ctx.SetResult(Result);
    end;
}
```

3. Enrol it. An install codeunit writes the catalog row in every company; the upgrade codeunit calls the same procedure, so a new version refreshes the description, schema and wrapper id. A new row starts with **Allow Run** off, and a re-run never

touches it, so the administrator's choice survives upgrades. Set **Is Mutating** honestly: off makes the verb run inside the forced rollback, which is right for a verb that only computes or renders.

```AL
codeunit 50101 "Contoso Act Install"
{
    // Enrols this app's actions in Wicket's Action Catalog. A new row starts with Allow Run off;
    // a re-run refreshes the fields this app owns and never touches Allow Run.
    Subtype = Install;

    trigger OnInstallAppPerCompany()
    begin
        Enrol();
    end;

    procedure Enrol()
    var
        Catalog: Record "Bcmag Action Catalog";
        ActionCode: Code[50];
    begin
        ActionCode := 'block-customer';
        if not Catalog.Get(ActionCode) then begin
            Catalog.Init();
            Catalog."Action Code" := ActionCode;
            Catalog."Allow Run" := false; // an administrator switches it on
            Catalog.Insert();
        end;
        Catalog."Wrapper Codeunit Id" := Codeunit::"Contoso Act Block Customer";
        Catalog.Category := 'customer';
        Catalog.Description := 'Set or clear a customer''s Blocked field. args: { systemId | no, blocked: none | ship | invoice | all }';
        Catalog."Is Mutating" := true;
        Catalog."Preview Supported" := false;
        Catalog.SetArgsSchema('{"type":"object","properties":{' +
            '"systemId":{"type":"string","format":"uuid","description":"The Customer SystemId; or send target.systemId"},' +
            '"no":{"type":"string","maxLength":20},' +
            '"blocked":{"type":"string","enum":["none","ship","invoice","all"]}},' +
            '"required":["blocked"],"oneOf":[{"required":["systemId"]},{"required":["no"]}]}');
        Catalog.Modify();
    end;
}
```

```AL
codeunit 50102 "Contoso Act Upgrade"
{
    Subtype = Upgrade;

    trigger OnUpgradePerCompany()
    var
        Install: Codeunit "Contoso Act Install";
    begin
        Install.Enrol();
    end;
}
```

4. Ship a permission set for the wrapper and the data it touches. A caller needs it together with `Wicket - Execute` (or above):

```AL
permissionset 50100 "Contoso Act"
{
    Assignable = true;
    Caption = 'Contoso Wicket actions';
    Permissions =
        codeunit "Contoso Act Block Customer" = X,
        tabledata Customer = RM;
}
```

5. Enable and call it. After installing the provider, an administrator reviews the wrapper, switches **Allow Run** on for `block-customer` on the Wicket Action Catalog page and assigns `Contoso Act`. Agents then find the verb in `capabilities.actions` with its `argsSchema`, and call it like any other action:

```
{ "data": {
    "action": "block-customer",
    "target": { "systemId": "<@systemId of a Customer row>" },
    "args":   { "blocked": "invoice" }
} }
```
JSON

The result is the wrapper's object, for example `{ "no": "10000", "blocked": "invoice", "changed": true }`.

Things to keep in mind when writing a provider:

- **Validate in the wrapper.** Wicket publishes the Args Schema to agents but does not check calls against it; the wrapper is the only thing between a malformed argument and the database.
- **Preview Supported** only for a wrapper that honours `args.preview` (see the Action Catalog table above). Leave it off otherwise.
- **Companies created later.** `OnInstallAppPerCompany` runs for the companies that exist at install time. To enrol in companies created afterwards, subscribe to the `Company` table's `OnAfterInsertEvent` and enrol there, as the companion does.
- **Review like a permission set.** The catalog row is the gate: once Allow Run is on, the wrapper bypasses the exposure list and the field policy, and can touch any table its code reaches.

13. Async delivery setup

1. Create a Storage account and a **private** container, for example `wicket-exports`.
2. On the setup card fill in **Azure Storage Account** and **Azure Blob Container**.
3. Choose **Azure Auth Mode** and run **Set Azure Credential**: paste the account key (Shared Key) or a SAS token with create, write, read and delete rights (Ready SAS). Under Ready SAS upload auth Wicket does not try to create the container, so it must already exist.
4. Choose **Download Mode**. `Service SAS` needs no further input. For `Ready SAS`, run **Set Azure Read SAS** and paste a read-only SAS token (`sp=r` , no `l`) without the leading question mark, with an expiry comfortably beyond Package TTL Hours; re-enter it before it gets that close to expiring, or new requests are refused.
5. Set TTLs and limits, then switch **Async Enabled** on.
6. Verify with `capabilities` : `async.enabled` is `true` and `async.downloadMode` names your choice.

The background worker runs as the caller who submitted the request, so the package respects that caller's exposure, field policy and BC permissions exactly as an inline read would; the table's exposure and readability are also checked when the request is submitted, so an unreadable target is refused at once rather than failing later in the worker. The package URL is a bearer capability: anyone holding it can download the package until it expires. Keep the container private and the SAS lifetime short.

14. Serving the usage guide

Host the single-file edition of the usage guide (`docs/wicket-usage-skill.md` in the plugin repository, davesmith7403/wicket — or your own edition) on any public HTTPS URL, or use the published copy directly: `https://raw.githubusercontent.com/davesmith7403/wicket/main/docs/wicket-usage-skill.md`, and enter it as **Skill Doc Url**. Agents without the MCP server then fetch it through the `skill` operation, whole or one `##` section at a time. Documents over 1 MB are refused; the URL must start with `https://`. Agents using the MCP server need none of this: the server bundles the skill (`wicket_skill`) and sends its essentials as MCP instructions.

15. Operating Wicket

TASK	WHERE	NOTES
See every call	**Wicket Operation Log** page	One row per request: operation, **Table Id**, **Target** (the codeunit and procedure for `invoke`, the action code for `action`, the async request for `status` / `cancel`), **Request Details** (while Log Request Details is on), caller, origin, logical status, cost, duration, problem type and detail, request id, idempotency key and **Unmasked Fields** (the `table:field` pairs the request read in clear although the baseline masks them; `query` and `readBlob` only). Read-only.

TASK	WHERE	NOTES
Purge old log rows	Automatic (the reaper), or a Job Queue Entry on codeunit 94121	Deletes Operation Log rows, idempotency snapshots and settled (Completed or Released) idempotency reservations older than **Operation Log Retention Days** (0 keeps everything); an In Progress reservation goes only once it is also past the stale window. The maintenance task armed at install runs it on every pass, so nothing needs setting up; the purge carries its own delete permission on those tables, so it works whichever user armed the maintenance task, and a failure is reported as telemetry `WICKET-0005`. A Job Queue Entry on codeunit 94121 still works for a schedule of your own; running both is harmless.
Watch async requests	**Wicket Request Queue** page	Status, progress, result size, the package's blob URL (never with a SAS token, since Audit users see every caller's rows), blob cleanup state, errors. The **Cancel** action stops a request.
Async queue retention	Automatic (the reaper)	Failed, Cancelled and Expired requests that finished longer ago than **Operation Log Retention Days** are deleted. A Succeeded request is never purged directly (it expires first), and a row whose blob deletion is still `Pending` is never purged. The purge carries its own delete permission on the queue, so it works whichever user armed the reaper; a failure is logged as telemetry `WICKET-0004`.
Telemetry	Application Insights	Event `WICKET-0001` per call with dimensions `operation`, `status`, `origin`, `cost`, `durationMs`, `problemType`; verbosity Normal below 400, Warning for 4xx, Error for 5xx. `WICKET-0002`: blob cleanup for a request gave up after the maximum number of delete attempts (12). `WICKET-0003`: one per finished async request (`requestId`, `status`, `packageFormat`, `resultRowCount`, `resultBytes`, `attemptCount`, `problemType`, `durationMs`; Warning when failed; never the query, URL or error text). `WICKET-0004`: the queue retention purge failed. `WICKET-0005`: the operation log retention purge failed.
See what the agent changed	**Wicket Changes** page (from the Setup card or the Operation Log)	One row per record the agent created, changed or deleted through Wicket, and per action or procedure it ran, with When, What, Table, Record, Fields, User and Connection; the part below lists each changed field, old → new. Values are stored as the field policy showed the field at its baseline (never as an unmask grant showed it) and checked again against today's policy when shown, so a field that has since become masked or hidden is shown that way; a stored mask is never shown in clear. Actions: the Operation Log entry, Business Central's **Change Log** for the record (knock-on changes in other records, such as the entries a posting creates, are there, not here), open the record, delete the filtered entries. Writes that fail or roll back leave no entry; an action or procedure that failed after it may have committed is listed as *Failed - may have changed data*. Requires `Wicket - Audit Changes`.
Change history retention	Automatic (the reaper)	Deletes entries older than **Change Retention Days**.
Grant read-only visibility	`Wicket - Audit`	Every admin page, no changes.
Rotate the storage credential	Setup card, **Set Azure Credential**	Replaces the stored secret; running requests finish with the old one.
Emergency stop for writes	Setup card, **Read-Only Mode**	Effective on the next call.

Idempotency snapshots store the full response of every idempotent call (the Response field is classified `CustomerContent`: it holds business data). An async acceptance is stored without the SAS token on its `packageUrl`. Retention is the same as for the Operation Log; keep it as short as your retry window allows.

16. Your first six calls

Everything below uses the same URL shape. Replace the placeholders once.

```
POST https://api.businesscentral.dynamics.com/v2.0/<tenant-id>/<environment>/api/wicket/rpc/v1.0/companies(<company-guid>)/wicketEndpoint(0)/Mic
Authorization: Bearer <token>
Content-Type: application/json
```

`wicketEndpoint(0)` is literal; `Microsoft.NAV.` is the mandatory action namespace. **Each envelope shown below is sent serialised, as the string value of the body's single `envelope` member** (`{"envelope": "{\"id\":\"first-1\",…}"}`), and the answer's `value` is a string you parse once (chapter 18). The repository file `docs/smoke-test.http` contains all eighteen requests ready for the VS Code REST Client.

1. capabilities — does the endpoint answer, and what does it allow?

```json
{ "id": "first-1", "issuedAt": "2026-09-04T09:00:00Z", "origin": "first-steps", "data": {} }
```

Expect `response.value.product = "Wicket"` , `transport.httpStatus = "always-200"` , eighteen entries in `operations` , and the live `limits` , `readOnly` , `fieldPolicy` , `async` and `actions` .

2. whoAmI — which principal and company did the token resolve to?

```json
{ "id": "first-2", "issuedAt": "2026-09-04T09:00:00Z", "origin": "first-steps", "data": {} }
```

Expect `{ userSecurityId, userId, company, readOnly, fieldPolicy, unmaskGranted, activeExceptions, strongEtag, serverTime }` . If `userId` is not the principal you registered, the token audience or the Entra Applications entry is wrong.

3. discover — what can I see?

```json
{ "id": "first-3", "issuedAt": "2026-09-04T09:00:00Z", "origin": "first-steps",
  "data": { "kind": "tables", "filter": "*Customer*", "top": 5 } }
```

Expect an entry with `id: 18, name: "Customer"` and `permissions: { read, insert, modify, delete }` . An empty list means the table is not exposed (chapter 10) or the principal lacks Read on it.

4. describe — what does the table look like?

```json
{ "id": "first-4", "issuedAt": "2026-09-04T09:00:00Z", "origin": "first-steps",
  "data": { "kind": "table", "tableId": 18 } }
```

Expect `fields` , `keys` , `primaryKey` , `hasSystemFields` and per-field `readable` / `writable` . Note which fields are listed: on a fresh install only `CustomerContent` and `SystemMetadata` fields appear.

5. query — read five rows

```json
{ "id": "first-5", "issuedAt": "2026-09-04T09:00:00Z", "origin": "first-steps",
  "data": { "tableId": 18, "select": ["No.", "Name"], "filter": "startswith(Name, 'A')", "top": 5 } }
```

Expect up to five rows, each with `No.` , `Name` , `@recordId` , `@systemId` and `@etag` . If more rows exist the response also carries `@nextLink` .

6. pageUrl — hand a record to a human

```json
{ "id": "first-6", "issuedAt": "2026-09-04T09:00:00Z", "origin": "first-steps",
  "data": { "table": "Customer", "systemId": "<an @systemId from step 5>" } }
```

Expect `{ pageUrl, pageId }`. Open the link in a browser: it lands on the customer card under your own login.

17. Reading a response

Every call returns HTTP 200. `response.value` is a string: parse it once.

```
result = json.parse(response.value)
if "type" in result and "status" in result:
    # RFC 7807 problem: { type, title, status, detail, instance, operation, suggest? }
    handle(result.type.slug, result.status, result.detail, result.suggest)
else:
    # the operation's result
```

PROBLEM FIELD	MEANING
`type`	`https://wicket.example/errors/<slug>`. Key your recovery on the slug.
`status`	The logical HTTP status: 400, 403, 404, 409, 412, 413, 422, 429, 500, 502, 503.
`title`	Short label.
`detail`	Plain English, including the fix.
`instance`	`urn:wicket:request:<id>` for correlation with the Operation Log.
`operation`	The operation that failed.
`suggest`	Optional machine hint: `{ field }`, `{ use }`, `{ kinds }`, `{ procedures }`, `{ tighten }`, `{ paging }`, `{ sections }`, `{ actions }`. Apply it programmatically.
extension members	A few problems carry extra members of their own, for example `replay-withheld`'s `originalStatus` and `reason`.

A real HTTP 401, 403 or 404 comes from the platform before Wicket runs: the token, the Entra Applications entry, the permission set on the endpoint page, or a malformed bound-action path.

18. Envelope and transport

On the wire. Business Central publishes a bound action only for simple parameter types (an action declared with a JSON-object parameter is silently left out of the API metadata), so every operation takes one text parameter and returns text:

```http
POST …/companies(<company-guid>)/wicketEndpoint(0)/Microsoft.NAV.whoAmI
Content-Type: application/json

{ "envelope": "{\"id\":\"a1\",\"issuedAt\":\"2026-09-24T00:00:00Z\",\"origin\":\"my-agent\",\"data\":{}}" }
```

The answer is `{ "@odata.context": "…#Edm.String", "value": "<result serialised as JSON>" }`. Text in `envelope` that is not one JSON object is `400 invalid-envelope`. Sending the envelope as an object instead of a string is refused by the platform before Wicket runs.

The envelope itself, before it is serialised:

```json
{
  "id":             "<uuid v4>",
  "issuedAt":       "<RFC 3339 UTC>",
  "origin":         "<your app tag, e.g. contoso.web/checkout@v3.2.1>",
  "idempotencyKey": "<optional, up to 128 printable characters>",
  "data":           { }
}
```

MEMBER	RULE
`id`	Required, non-empty. Echoed as `@requestId`.
`issuedAt`	Required, RFC 3339 UTC.
`origin`	Required, up to 256 characters from `[A-Za-z0-9._@/+-:]`. Stamped on the Operation Log.
`idempotencyKey`	Optional for most operations; **required** for `invoke` and `action` while Setup **Require Key For Actions** is on (the fresh-install default), else `400 idempotency-key-required`. Up to 128 printable characters; the rules follow below.
`data`	Required object; the operation payload.

There are no request-header semantics: a bound action cannot read headers, so the idempotency key and the etag both travel in the body. A malformed envelope fails with `400 invalid-envelope` before any operation runs.

Idempotency keys

A key names **one logical request**. Send a fresh unique key per request and reuse it only to retry that same request.

- **Replay.** A resend with the same key and the same `{operation, data}` returns the stored response with `"@replayed": true` (the original `@requestId` intact) and executes nothing. The same key with a different body is `409 idempotency-conflict`, also when the earlier attempt with that key was released for a retry.
- **At most once.** The key is reserved before execution. A resend while the first request is still running gets `409 idempotency-in-progress`: wait, then resend. An earlier attempt whose outcome cannot be known (its session died, or no snapshot survived) gets `409 idempotency-indeterminate` and is never re-run: query the affected records, then use a new key.
- **What is stored.** Final outcomes only: 2xx, and 4xx that describe the request itself (malformed input, a missing record, a business failure such as `422 action-failed`, where a posting may have committed). Never stored, and the key released for a retry with the **same** key: every 5xx on the read and write planes (including a commit-time failure, reported as `500 internal`, in which none of the operation's changes took effect), and the pre-execution refusals an administrator can lift: `read-only`, `role-forbidden`, `not-exposed`, `invoke-disabled`, `action-disabled`, `trigger-bypass-disabled`, `async-not-configured`, `too-many-requests`. `idempotency-key-required` is not stored either: there is no key to store it under. A 5xx from `invoke` or `action` is the exception: the wrapper may already have

committed (a posting commits inside the call), so the key is settled rather than released, and a resend with the same key answers `409 idempotency-indeterminate`: check the business result before using a new key.

- **Re-authorized replay.** A stored response is handed out only if the caller could still obtain it now: read-only mode, role rung, the target's exposure and BC permission, the field policy for every field the stored rows carry (a stored row that carries a field now masked or hidden is withheld, `field-masked` or `field-restricted`), and, for a stored `query`, the fields it was filtered, ordered or windowed by. A row whose table cannot be determined fails closed. When the check fails for a stored **success** (2xx), the resend gets **`403 replay-withheld`** with members `originalStatus` (the status the original request completed with) and `reason` (the slug of the refusal, for example `read-only` or `field-restricted`). Its `detail` states that the original request **completed**. Do not repeat it under a new key; that would run it twice. The same key replays again once access is restored. A stored 4xx took no effect, so its resend simply gets the refusal a fresh call would get now.
- **Scope and retention.** Keys are scoped to the calling principal. Snapshots and reservations are purged with the Operation Log. An async acceptance is stored without the SAS token on its `packageUrl`; read a signed URL from `status`.

19. discover and describe

discover

```
{ "data": { "kind": "tables", "filter": "*Sales*", "top": 200, "languages": ["en-US", "da-DK"] } }
```
JSON

- `kind`: `tables`, `codeunits`, `companies` or `apis`.
- `filter`: an **AL filter string** on the object caption (the company name; the entity set name for `apis`), not the OData filter `query` uses. `*Sales*` matches any caption containing Sales.
- `top`: defaults to Default Top, capped at Max Top.
- `languages`: up to five culture tags, each a string (not a comma list); more, or a non-string entry, is `400 invalid-field`. The same rule applies to `describe`.
- `publisher`: an AL filter on the publisher of the app that owns each table or codeunit. `"<>Microsoft"` lists solution and custom objects, `"Contoso*"` one vendor's. Platform objects that belong to no installed app never match it. A malformed filter is `400 invalid-filter`.
- Result rows: `{ id, name, captionML, scope, app, permissions }`. `app` is `{ id, name, publisher, version }` for the installed app that defines the object, or `null` for a platform object (system and virtual tables). Tables appear only when exposed and readable (Wicket's audit and queue tables never appear, `Open` mode included); codeunits only when invoke-enrolled (`permissions: { invoke: true }`); companies as `{ id: <name>, name, captionML, scope: "company" }`.

`kind: "apis"` lists the tenant's own API pages and API queries: `{ id, scope: "api", apiKind: "page"|"query", name, entitySetName, entityName, route, routes, sourceTableId?, operations }`. `route` is what goes between `/api/` and `/companies(...)`, plus the entity set: `v2.0/customers` for Microsoft's standard APIs, `<publisher>/<group>/<version>/<entitySet>` for anything else; `routes` lists every declared version, and `route` uses the highest. The answer also carries `catalogVersion`, `apiRouting`, `@count` and, while more rows exist, `@nextSkip`. Extra members: `tableId` (only the API pages over one table), `skip`, and `ifNoneMatch`.

Caching the catalog. `catalogVersion` fingerprints the catalog's content — the API pages and queries themselves, plus every installed app's package when the caller may read that — so it moves when an app is installed, upgraded or removed, and not when Wicket's policy changes. Cache the catalog under it and revalidate by sending the cached value as `ifNoneMatch`: an unchanged catalog answers `notModified: true` with no rows. `capabilities.apiRouting.catalogVersion` carries the same value. The MCP server's `wicket_apis` does this for you, in memory and on disk. The catalog is not filtered by the exposure list, because which APIs exist is already public in the tenant's `$metadata`; Wicket's own endpoint is never listed.

describe

```
{ "data": { "kind": "table", "tableId": 18 } }
{ "data": { "kind": "codeunit", "codeunitId": 94190 } }
```
JSON

For a table: `{ tableId, name, captionML, schemaVersion, primaryKey: [fieldNo], fields: [...], keys: [...], hasSystemFields, permissions }`. Each field: `{ no, name, type, length, class, dataClassification, captionML, nullable, readable, writable, access, binary?, tableRelation?, optionMembers? }`. `access` is `"clear"` or `"masked"`; a masked field also carries `mask: { profile, kind }`, `filterable: false` and `sortable: false`, and its `type` is the type a masked value is emitted as (a string for Hash and Fixed Text, except Hash on a Boolean or Option/Enum field, which is emitted as `null` with its own type and `mask.kind` `null`). Chapter 11.

- `type` and `class` are invariant names (`Decimal`, `Code`, `Normal`, `FlowField`), the same in every session language.
- `binary: true` marks BLOB, Media and MediaSet fields; read them with `readBlob`.
- `optionMembers` on Option and Enum fields lists `{ ord, name, captionML }`. A query row's option value `{ ord, name }` carries the same invariant `name`, so you can match on either.
- `hasSystemFields: false` means the table carries no `@etag` and writes are last-write-wins unless Require ETag On Writes is on.
- Only `Normal` and `FlowField` classes are listed.

A table's describe also lists `apiRoutes: [ { pageId, entitySetName, route, sourceTableView? } ]`, the API pages built on it.

A table's describe also carries `app` (the owning installed app, as in `discover`) and **`describeVersion`**, a SHA-256 hash of the answer as this caller sees it. It moves with anything that changes that view: the table's structure, the field policy and masking, the caller's exceptions and unmask tier, and their Business Central permissions. Send a cached value back as `ifNoneMatch` and an unchanged answer is `{ tableId, describeVersion, notModified: true }` with no schema. So a client can cache a table's describe and revalidate it on every use without ever serving a stale policy view.

```
{ "data": { "kind": "table", "tableId": 18, "include": { "referencedBy": true } } }
```
JSON

`include.referencedBy` adds **`referencedBy: [ { tableId, tableName, fieldNo, fieldName } ]`**: the fields of other tables (and self-references) whose table relation targets this table. It is how a solution table that hangs off Customer is found. Only tables the caller may read (exposure and BC read permission) and fields the field policy does not hide are listed, so a hidden field's name never appears. The list stops at 200 entries and then sets `referencedByTruncated: true`. It scans the field metadata of every table, so ask for it when you are exploring, not on every call.

For a codeunit: `{ codeunitId, name, schemaVersion, procedures: [ { procedure, returnShape, argsSchema?, description? } ] }`, enrolled procedures only.

```
{ "data": { "kind": "api", "pageId": 30009 } }
```
JSON

For an API page (`pageId`, or `queryId` for an API query): the catalog row plus `sourceTableView`, `fields: [ { name, mapped, fieldNo?, tableId?, readable } ]`, `routable`, `notRoutableReason?` and `catalogVersion`. Each API field names the table field it is bound to; one with `mapped: false` is computed by the API's own code and is never `readable`, because the field policy cannot judge it. `routable` is true only when API Routing is `Prefer` and Wicket would let the caller read the source table (exposure list, BC permissions); otherwise `notRoutableReason` says `routing-off`, `not-exposed` or, for an API query, `no-field-mapping`. The answer does not refuse a table the caller may not read (the API's shape is public); it answers `routable: false`. These governance answers follow the policy as it is now, so a client asks again before every API read. `sourceTableView` matters when choosing: an API may cover only part of its table (sales orders but not quotes, say). An id that is not an API page is `404 unknown-api`.

Reading through an API (MCP)

`wicket_api_read { pageId, select?, filter?, orderBy?, top?, skip?, count? }` is the MCP server's API-first read. It asks `describe kind:"api"` first, refuses unless `routable`, and allows only `readable` fields in `select`, `filter` and `orderBy`; with no `select` it selects exactly the readable fields and drops anything else the API returns. Navigation and lambdas (`/`) in a filter are refused, and so is a field name the API does not have. Refusals carry `suggest: { use: "wicket_query", tableId }`. Use it when an API page covers the table and the fields you need; use `query` when routing is `Off`, when the fields are not in the API or not readable, or when the user asks for dynamic access.

20. query

```json
{ "data": {
    "tableId": 18,
    "select":  ["No.", "Name", "Balance (LCY)"],
    "filter":  "\"Balance (LCY)\" gt 1000 and Blocked eq ' '",
    "orderBy": ["Name asc"],
    "top":     50,
    "include": { "flowFields": true },
    "count":   true
} }
```

The result is an array of rows. Each row carries the selected fields plus `@recordId`, `@systemId` and (when the table has system fields) `@etag`. Alongside the array: `@nextLink` when more rows exist, `@count` when `count: true`, and `@pagingMode` (`keyset` without `orderBy`, else `offset`; see Paging).

How values arrive: Text and Code values verbatim (a Text value's leading and trailing spaces are data and are not trimmed; an empty value is `null`); Option and Enum values as `{ ord, name }`, where `name` is the invariant member name that `describe` also reports (empty when the stored ordinal is no longer a member); DateFormula, RecordId and TableFilter values as their text form; dates, times and date-times in ISO 8601 (UTC for date-times); large decimals as strings (Decimal String Threshold).

Members

MEMBER	NOTES
`select` / `selectFieldNo`	Field names or numbers; one or the other. Omit both and you get only the primary key. `["*"]` alone means every readable non-binary field.
`filter`	The OData subset below.
`orderBy`	`["Name asc"]`. One direction for all terms; the fields must match an existing key, otherwise `400 invalid-orderby`. The platform sorts by the whole key it picks, so ordering by a key whose later field is restricted by the field policy is `403 field-restricted` (the detail names only the fields you supplied). Omit it to sort by primary key, which also gives keyset paging.
`top`, `skip`, `cursor`	Page size, manual offset, or the opaque `@nextLink` from the previous page. A cursor that does not decode, or was issued for another table or another query shape, is `400 invalid-cursor`.
`count`	Adds `@count`, the total for the filter.
`include.flowFields`	FlowFields are computed only when this is true and the field is selected.
`modifiedAfter`, `modifiedBefore`	ISO 8601 window on `SystemModifiedAt`, ANDed with `filter`. The incremental-sync pattern. Do not also filter on `SystemModifiedAt`; an inverted window is refused.
`aggregate`	Server-side summary instead of rows; see below.
`deliver`	Async delivery; see chapter 24.
`languages`	Culture tags for `captionML` values; at most five.

Filter cookbook

YOU WANT	WRITE
Equality, inequality	`Status eq 'Open'`, `Status ne 'Closed'`
Numeric, date, datetime comparison	`Balance gt 1000`, `"Posting Date" ge 2026-01-01`, `SystemModifiedAt gt 2026-01-01T00:00:00Z`

YOU WANT	WRITE
AND chaining	`Country eq 'US' and Blocked eq ' ' and Balance gt 0`
Same-field OR	`"No." in ('C-1','C-2','C-3')` (at most 20 members, comma-separated, no trailing comma, no `null`)
Prefix, suffix, substring	`startswith(Name,'Acme')`, `endswith(Name,'ltd')`, `contains(Name,'co')`
Blank, non-blank	`"VAT Registration No." eq null`, `ne null`
Boolean	`"Privacy Blocked" eq true`
Apostrophe in a value	`Name eq 'O''Brien'`
Field name with spaces or punctuation	`"Customer Posting Group" eq 'DOMESTIC'`
Field by number	`#22 gt 1000`

Not supported, with specific errors: cross-field `or`, standalone `not`, arithmetic, lambdas, casts. Comparisons and `in()` are exact (a `*` is a literal star). Each `in()` member is converted to a typed value by its literal kind (numbers, dates and date-times parsed invariantly, as in a single comparison), so `Amount in (1.5, 2.5)` means the same under every regional setting. A missing or trailing comma, or `null`, inside `in()` is `400 invalid-filter`; test for blank with `eq null`. Inside `startswith`, `endswith` and `contains`, `*` and `?` are live Business Central wildcards. Option fields take the member name as a string, and the blank `Blocked` state is a single space.

Paging

1. Send the query with `top`.
2. Read the rows. If there is no `@nextLink`, stop.
3. Send the same query again with `data.cursor` set to the `@nextLink` value.
4. Repeat.

The cursor is opaque; pass it back verbatim with the same `tableId`, `filter` and `orderBy`. `@pagingMode` says which of two mechanisms is in use:

- **`keyset`** (no `orderBy`). Rows come in primary-key order and the cursor carries the last returned row's key position; the next page starts strictly after it. No row is re-read, so every page costs only its own rows against the scan budget however deep the traversal goes, and an insert or delete between pages cannot shift it. Prefer it for full traversals.
- **`offset`** (with `orderBy`). The cursor carries a row offset. Skipped rows are scanned again and count against the scan budget, so a deep page can hit `422 scan-budget` on its own (`suggest.paging` then recommends dropping `orderBy`), and concurrent inserts can shift rows between pages. Filter to a fixed window when stability matters.

A malformed cursor is `400 invalid-cursor`, never a silent restart at page one. Cursors issued before keyset paging existed are still honoured.

Aggregates

```json
{ "data": {
    "tableId": 21,
    "filter":  "\"Customer No.\" eq 'C-1234'",
    "aggregate": { "count": true, "sum": ["Amount"], "avg": ["Amount"], "min": ["Posting Date"], "max": ["Posting Date"] }
} }
```

The result is one object: `{ count, sum: { Amount }, avg: { Amount }, min: { "Posting Date" }, max: { "Posting Date" } }`. `aggregate` composes only with `filter` and the modified window; `sum` and `avg` need numeric `Normal` fields; every named field must be readable. On an empty set `count` is 0, `sum` is 0 and the others are `null`. A total the server cannot compute, typically an Integer column whose sum passes 2,147,483,647, is `422 aggregate-overflow` with `suggest.tighten` (narrow the filter or aggregate in slices); an overflow raised inside the database itself can surface as `500 internal`.

Limits that bite

LIMIT	DEFAULT	ERROR
Rows examined per page	100 000	`422 scan-budget`, with `suggest.tighten` (and `suggest.paging` under `orderBy`)
`top`	100 default, 1000 max	Silently capped
Filter length	2048	`413 payload-too-large`
Selected fields	256	`413 payload-too-large`
`in()` members	20	`400 invalid-filter`
`languages`	5	`400 invalid-field`
`readBlob` size	10 MB	`413 blob-too-large`

21. readBlob

```json
{ "data": { "tableId": 18, "recordId": "<@recordId>", "fieldNo": 8001 } }
```

Returns `{ contentType, bytesBase64 }`. The content type is the stored MIME type or one sniffed from the bytes (PNG, JPEG, GIF, PDF, ZIP, SVG), else `application/octet-stream`. Take the field number from `describe` (`binary: true`).

The field policy is checked first, before the record is looked up, and problems name the field by number only. A hidden field and a field number that does not exist both answer `403 field-restricted`, so a probe cannot tell hidden from absent; a binary field the caller may read only masked answers `403 field-masked`; a readable non-binary field is `400 not-binary`. An empty field returns `404 empty-blob`, a MediaSet field `422 mediaset-unsupported`, and a binary larger than 10 MB `413 blob-too-large` (the bytes travel base64-encoded inside one JSON response).

22. insert, modify, delete, batch

insert

```json
{ "data": {
    "tableId": 18,
    "values":  { "No.": "C-1234", "Name": "Acme Ltd.", "Country/Region Code": "US" },
    "init":    true
} }
```

`values` (by name) or `valuesByFieldNo` (by number), not both. `init` (default true) runs the insert trigger; `init: false` skips it and is refused with `403 trigger-bypass-disabled` unless Setup **Allow Trigger Bypass** is on. Values are coerced to the AL type and applied with `Validate`, in the order given, so send fields in validation order (for a sales line: Type, then No., then Quantity). Option values accept a member name, an ordinal, `{ "ord": n }` or `{ "name": "..." }`; `null` clears a field. Result: `{ recordId, @etag? }`. Failures: `422 validation` with `suggest.field`, or `422 insert-failed` with the Business Central error text.

modify

```json
{ "data": {
    "tableId":  18,
    "recordId": "<@recordId>",
    "etag":     "<@etag from the same read>",
    "values":   { "Blocked": " " },
    "validate": true
} }
```

The record is re-located under the caller's security filter and locked, then the concurrency rule applies:

SITUATION	OUTCOME
`etag` matches	Write proceeds; the result carries the new `@etag`.
`etag` stale	`412 precondition-failed`. Re-read, re-apply your change, retry.
`etag` omitted, table has system fields	Last-write-wins, unless Require ETag On Writes is on: then `409 etag-required`.
Table has no system fields, tenant requires etag	`409 no-etag-available`.

`validate` (default true) runs each field's OnValidate; `validate: false` is refused with `403 trigger-bypass-disabled` unless Allow Trigger Bypass is on. A `modify` never changes a primary-key field: a change to any key field, by name, number or as a side effect of another field's validation, is `400 pk-immutable` (a key change is a rename, which Wicket does not offer; insert a new record and delete the old one).

delete

```json
{ "data": { "tableId": 18, "recordId": "<@recordId>", "etag": "<@etag>" } }
```

Same concurrency rules. Result: `{ deleted: true, recordId }`.

batch

```json
{ "data": { "preview": true, "items": [
  { "op": "insert", "tableId": 36, "values": { "Document Type": "Order", "Sell-to Customer No.": "10000" } },
  { "op": "insert", "tableId": 37, "values": { "Document Type": "Order", "Document No.": { "fromItem": 0, "field": "No." },
                                               "Line No.": 10000, "Type": "Item", "No.": "1896-S", "Quantity": 2 } },
  { "op": "modify", "tableId": 18, "recordId": "<@recordId>", "etag": "<@etag>", "values": { "Credit Limit (LCY)": 75000 } }
] } }
```

Up to 100 items (`limits.maxBatchItems`), any tables, each in the shape of `insert`, `modify` or `delete` plus `op`. Every item runs through the same handler as a single write, so exposure, the field policy, validation, the etag check and the change history apply to each; the items run in one transaction, with explicit commits inside table triggers ignored, so **all are written or none**. The first failing item stops the batch: the problem is that item's own (`400`, `403`, `412`, `422` …) with the extension members `item` (its index) and `itemOp`, and a `detail` starting `items[1] (insert, table 37):`; nothing is written. `preview: true` runs every item inside a forced rollback and answers the per-item results (their recordIds and etags describe records that no longer exist) or the first failure; it is allowed in Read-Only Mode.

A value `{ "fromItem": n, "field": "<name>" }` (or `"fieldNo"`) is that field of the record item n wrote, read after it ran — the header's number in the example above. `recordId: { "fromItem": n }` targets the record item n created or changed. Only earlier items; a referenced field must be readable in clear (`403 field-restricted` / `field-masked` otherwise, so a hidden value is never copied into a visible field); a later item, a deleted record or an unknown field is `400 invalid-reference`. Result: `{ items: [ { index, op, recordId, @etag? | deleted } ], count, preview? }`; cost is the number of items. Requires `Wicket - Edit`. An idempotency key covers the whole batch; a replay re-checks every item's table.

Making writes safe to retry

Put an `idempotencyKey` on every write your client might resend. The first execution stores its outcome; a resend with the same key and body replays it, so a network timeout after a successful insert cannot create a duplicate. A failed write rolls back completely, including side effects of table triggers, because the handler raises inside the `Codeunit.Run` boundary. A replay is not a bypass: it is checked against the policy in force at the moment of the resend, so a caller whose access was narrowed in between receives `403 replay-withheld`, which says the write **did** complete and must not be repeated under a new key (chapter 18).

23. invoke and action

invoke

```json
{ "data": { "codeunitId": 94190, "procedure": "Ping", "args": { "message": "hi" } } }
```

Runs an enrolled wrapper. Result: the wrapper's object, or `{ ok: true }` for a void return. `args`, when present, must be a JSON object (`400 invalid-field` otherwise). Failures: `400 idempotency-key-required` (no key while Require Key For Actions is on), `403 not-exposed`, `404 unknown-procedure` (with `suggest.procedures`; also for a procedure name longer than 120 characters), `403 invoke-disabled`, `422 invoke-failed` with the Business Central error text. Synchronous; a long posting blocks the call.

action

```json
{ "data": {
    "action":  "post-sales",
    "target":  { "systemId": "<@systemId of a Sales Header row>" },
    "args":    { "ship": true, "invoice": true },
    "preview": false
} }
```

`action` is the discoverable front door over the same wrappers. Read `capabilities.actions` first: it lists every enrolled verb as `{ action, enabled, category?, mutating, previewSupported, description?, argsSchema? }`. Call only verbs with `enabled: true`; a disabled one answers `403 action-disabled`, and `argsSchema` names the arguments. Send an `idempotencyKey` (required while Require Key For Actions is on).

- `target.systemId` is merged into the wrapper arguments as `systemId`. An `args.systemId` naming a **different** record is `400 invalid-field`; send one of the two.
- `args` must be a JSON object when present (`400 invalid-field`).
- `args.preview` belongs to the framework: request a dry run with `data.preview: true`, which always sets `args.preview` to true for the wrapper. An `args.preview` without `data.preview: true` is `400 invalid-field`.
- **Two boundaries.** A mutating verb's wrapper runs and its writes are committed (a posting commits on its own anyway). A preview, and every verb enrolled with Is Mutating off, runs inside a **forced rollback**: explicit commits are ignored and every write is undone when the wrapper returns (for a non-mutating verb a nested `Codeunit.Run` that would commit is refused too), and only the result comes back. Because nothing persists in Business Central, exactly those calls are allowed on a read-only endpoint and need no idempotency key. Chapter 12 lists what no rollback can undo.
- An action code longer than 50 characters is simply `404 unknown-action`.

Results are verb-specific:

VERB	RESULT
`post-sales`	`{ posted: true, sourceDocumentType, sourceNo, postedShipmentNo?, postedReturnReceiptNo?, postedInvoiceNo? }`
`post-purchase`	`{ posted: true, sourceDocumentType, sourceNo, postedReceiptNo?, postedReturnShipmentNo?, postedInvoiceNo? }`
`post-gen-journal`	`{ posted: true, template, batch, glRegisterNo, firstEntryNo, lastEntryNo }`, or `noEntriesPosted: true` in place of the register fields when th
`post-item-journal`	`{ posted: true, template, batch, itemRegisterNo, firstEntryNo, lastEntryNo }`, or `noEntriesPosted: true`
`release-sales`, `release-purchase`	`{ released: true, sourceDocumentType, sourceNo, status, changed }`
`reopen-sales`, `reopen-purchase`	`{ reopened: true, sourceDocumentType, sourceNo, status, changed }`
`make-order-from-quote`, `make-invoice-from-quote`, `make-purchase-order-from-quote`	`{ converted: true, sourceDocumentType: "Quote", sourceNo, quoteDeleted, createdDocumentType, createdNo, createdSystemId }`
`document-pdf`	`{ document, no, fileName, contentType: "application/pdf", sizeBytes, base64 }`, refused above 10 MB
`item-availability`	`{ itemNo, description, baseUnitOfMeasure, locationCode, variantCode, date, inventory, grossRequirement, scheduledReceipt, availab

`status` is the invariant status name after the call (`Open` , `Released` , `Pending Approval` , `Pending Prepayment`); `changed` is false when the document already had that status. `createdSystemId` is in the `@systemId` spelling, ready to pass as `target.systemId` to the next verb. `document-pdf` renders the report the tenant's **Report Selections** choose for that document usage (and the customer's or vendor's document layout), as the document's Print action would. `item-availability` uses Business Central's **Available to Promise** with the company's *Check-Avail. Period Calc.* and *Time Bucket*, as order promising does; `earliestAvailableDate` (with `quantity`) is `null` when the quantity is not available within that look-ahead.

Previewing a posting. `preview: true` on any of the four posting verbs runs Business Central's own posting preview (the engine behind the *Preview Posting* action) and returns what the post would create, without posting:

```json
{ "preview": true, "wouldPost": true, "errors": [],
  "entries": [ { "table": "G/L Entry", "tableId": 17, "count": 3, "debit": 125, "credit": 125 },
               { "table": "VAT Entry", "tableId": 254, "count": 1, "base": -100, "amount": -25 },
               { "table": "Detailed Cust. Ledg. Entry", "tableId": 379, "count": 1, "amount": 125 } ],
  "glAccounts": [ { "no": "2310", "debit": 125, "credit": 0 }, { "no": "5610", "debit": 0, "credit": 25 }, { "no": "6110", "debit": 0, "credit":
  "sourceDocumentType": "Invoice", "sourceNo": "S-INV-1001" }
```

- `entries` has one row per ledger with its count and the amounts held in normal fields: G/L `debit` / `credit` , VAT `base` / `amount` , detailed customer and vendor entries and bank entries `amount` , item ledger `quantity` , value entries `costAmount` / `salesAmount` . `table` is the invariant table name. `glAccounts` is the debit and credit per G/L account.
- A post that would fail is still a completed call: `wouldPost: false` and `errors` with Business Central's reasons (in the session language), and no entries. Fix the document and preview again.
- The caller's `ship` / `invoice` / `receive` selection is applied to the preview. Journals take `template` and `batch` as for the post, after the same guards.
- The preview runs inside the forced rollback, so it is allowed on a read-only endpoint and without an idempotency key, and the posting number series is not consumed. A real post afterwards creates the entries the preview showed, unless the data changed in between.

A posted-number member appears only when that phase happened in this call. For a **credit memo or return order**, `postedInvoiceNo` holds the posted **credit memo** number (the member name is kept for compatibility). Return orders default to receive + invoice (sales) and ship + invoice (purchase); explicit arguments always win. The journal evidence is the register created above the register number held before the post, matched on template, batch and the calling user for the G/L register (batch names repeat across templates) and on batch and the calling user for the item register (which has no template field), so a concurrent posting in another session is not reported as this one. When an error follows the post, the wrapper claims `posted: true` only if exactly one such register exists.

Posted despite an error. The posting routines commit the posted document and then run more code (analysis-view refresh, subscribers to the after-posting events). If that later code fails, the document **is** posted. The wrapper then re-reads the database and, when it finds proof (the source header's posting-number stamps moved, or a posted document for the source created during this call, or for journals a new register for the batch), answers `posted: true` plus `postCommitWarning` (the error text) and `postCommitEvidence` (what proved the posting) instead of a failure. Treat that as a success that needs a human look; never post the document again.

SLUG · STATUS	MEANING
`unknown-action` · 404	Not enrolled or no provider installed.
`action-disabled` · 403	Enrolled, `Allow Run` off.
`read-only` · 403	Read-Only Mode is on and the call would mutate. A preview or a non-mutating verb still runs.
`preview-unsupported` · 400	`preview: true` on a verb without preview support (`previewSupported: false`).
`action-failed` · 422	The wrapper raised an error; `detail` carries the posting message.
`idempotency-key-required` · 400	No envelope key while Require Key For Actions is on.

Posting commits immediately and is not covered by Wicket's rollback. Treat a mutating action as the last write of its request. Sales-Post and Purch.-Post also commit in stages: work committed before a later failure stays committed even when the answer is `422 action-failed`. The document may have been released, had its posting numbers assigned, or (an order) been partly shipped or received. Re-read the document before trying again, and remember that a `422` is the key's final, stored answer: a retry after fixing the document needs a **new** key. The calling principal needs the standard Business Central posting permissions in addition to `Wicket Act`.

24. Async delivery

Add a `deliver` block to a `query` and the rows are written as a package to Azure Blob Storage instead of returned inline. The worker pages internally, so async delivery escapes the per-call scan budget.

```json
{ "data": {
    "tableId": 21,
    "select":  ["*"],
    "filter":  "\"Posting Date\" ge 2026-01-01",
    "deliver": { "mode": "azureBlob", "package": "ndjson", "scheduleAt": "2026-09-05T02:00:00Z" }
} }
```

MEMBER	NOTES
`mode`	Required, `azureBlob`.
`package`	`ndjson`, `json` or `zip`; defaults to the tenant's Default Package Format.
`scheduleAt`	Optional RFC 3339 string. Omit for immediate execution (`null` or a non-string is `400 invalid-field`); set for one deferred run within Max Schedule Ahead Days. No recurrence.

The submit is validated before anything is queued: the `deliver` members must have the right JSON types, and `tableId` must name a table this caller may read right now (`400 missing-field` without it, `403 not-exposed` when it is not exposed or not readable), so an unreadable target is refused at once instead of failing later in the worker.

The call returns at once:

```json
{ "value": {
    "@async": true, "requestId": "9f1c…", "status": "queued",
    "packageUrl": "https://acct.blob.core.windows.net/wicket-exports/9f1c….ndjson",
    "scheduledFor": null, "expiresAt": null,
    "statusVia": { "operation": "status", "data": { "requestId": "9f1c…" } }
} }
```

An immediate request returns its `packageUrl` up front, but the URL answers 404 until the worker has written the package. A scheduled request withholds the URL until `status` reports `succeeded`. `expiresAt` is `null` at acceptance: retention starts when the request finishes, and `status` reports `expiresAt` (= `completedAt` + Package TTL Hours) from then on. If the platform cannot start a background worker (task or session capacity), the submit answers `503 async-start-failed` and the request is recorded as failed, so it does not hold a slot of your quota; retry shortly.

The URL in a response is signed when the response is built. A replay of the acceptance under the same idempotency key carries the bare blob URL without its SAS token; call `status` for a signed one.

Lifecycle

`queued` or `scheduled` → `running` → `succeeded` | `failed` | `cancelled`, and finally `expired` when the package TTL passes and the blob is deleted.

```json
{ "data": { "requestId": "9f1c…" } }                // status
{ "data": { "status": "running", "top": 20 } }       // requests
{ "data": { "requestId": "9f1c…" } }                // cancel
```

`status` returns `{ requestId, status, operation, packageFormat, createdAt, scheduledFor, startedAt, completedAt, expiresAt, progressRows, resultRowCount, resultBytes, packageUrl?, problem? }`.

`packageUrl` appears once the request has `succeeded` and is composed on every call: under Service SAS it carries a freshly minted token (https only, valid from 15 minutes back until now + SAS TTL Minutes, never past `expiresAt`), so a `status` read hours after completion still yields a working link while the package lives. `requests` returns an array of the same projection, newest first. `cancel` finalises a scheduled or queued request (not yet started) at once, and asks a running one to stop at its next checkpoint; a worker that finishes its upload after the cancel honours it, and the package is deleted. A failed request's `problem.type` names the cause: `async-upload-failed` (the package could not be written to Azure, or the download configuration became unusable), `async-task-failed` (the scheduled task itself failed), `async-stale` (the worker session ended without finishing), `async-start-failed`, or the query's own problem.

All three are strictly caller-scoped: another principal's request id is indistinguishable from a nonexistent one (`404 unknown-request`). Backpressure: more than Max Active Requests open requests return `429 too-many-requests`. An `idempotencyKey` on the submit replays to the same `requestId` instead of spawning a duplicate export.

25. capabilities, skill, pageUrl, whoAmI, changes

capabilities

`{ "data": {} }` returns `{ product, apiVersion, schemaVersion, urlTemplate, transport, envelope, operations, filterGrammar, limits, readOnly, structureVersion, strongEtag, exposureMode, fieldPolicy, skill, async, actions, paging }` (and `apiRouting`). `structureVersion` is a SHA-256 over the installed apps' packages: it moves when an app is installed, upgraded or removed, which is when table structure can have changed, and not when Wicket's policy changes. A client's structural knowledge of the tenant (a schema cache, the MCP server's tenant map) should be re-verified when it moves. `exposureMode` is the invariant name (`Allowlisted` for Exposed tables only, or `Open`); `fieldPolicy` is `{ mode, classifications: [{ name, access, maskProfile }], maskProfiles, unmaskTierEnabled, principal: { unmaskGranted, activeExceptions }, describeRestrictedFields }`, so an agent learns what it will see masked and whether its own grants lift it; `paging` describes the keyset and offset modes. Call it once at startup and configure your client from it rather than hard-coding limits.

skill

```json
{ "data": { "mode": "toc" } }
{ "data": { "mode": "section", "section": "query" } }
{ "data": { "mode": "full" } }
```

`toc` returns `{ title, description?, sections[] }`; `section` returns `{ section, content }` for one H2 heading (matched case-insensitively, exact then substring); `full` returns `{ content }`. The document is whatever the administrator hosts at Skill Doc Url — the repository's single-file edition is `docs/wicket-usage-skill.md`. Failures: `404 skill-not-configured`, `502 skill-url-invalid` / `skill-fetch-failed` / `skill-too-large`, `404 unknown-section` with `suggest.sections`, `400 invalid-mode`.

pageUrl

```json
{ "data": { "table": "Customer", "systemId": "<@systemId>" } }
{ "data": { "table": "#36", "recordId": "<@recordId>", "pageId": 42 } }
```

`table` is a name or `#<number>`. Pass `systemId` or `recordId`; `systemId` wins when both are present. `pageId` overrides the table's default lookup page and is required when the table has none (`404 no-page`). Requires `Allow Read` on the table. The link opens under the viewer's own login and permissions.

whoAmI

`{ "data": {} }` returns `{ userSecurityId, userId, company, readOnly, fieldPolicy, unmaskGranted, activeExceptions, strongEtag, serverTime }`; `unmaskGranted` says whether the Unmask tier lifts masks for this caller right now

and `activeExceptions` counts the caller's active per-user exceptions, which answers "why do I see this value in clear?" without admin access. Use it after minting a token or switching connections, and use `serverTime` as the watermark for incremental sync.

changes

```json
{ "data": {} }
{ "data": { "from": "2026-10-01T00:00:00Z", "tableId": 18, "kind": "changed", "top": 50 } }
{ "data": { "user": "ANNA", "viaConnection": "contoso-sandbox", "cursor": "<@nextLink>" } }
```

Returns `{ value: [ { entryNo, createdAt, kind, tableId, tableName, recordKey, systemId?, user, connection, requestId, name?, outcome?, result?, fieldCount, fields: [ { fieldNo, fieldName, shownAs, old?, new?, truncated? } ] } ], @nextLink? }`, newest first; `kind` is `created`, `changed`, `deleted`, `action` or `procedure`, and `shownAs` is `clear`, `masked` or `hidden` (no values). A caller sees its own entries; one holding `Wicket - Audit Changes` sees everyone's and may filter by `user` (`403 changes-forbidden` otherwise). `top` defaults to 20, capped at 100; pass `@nextLink` back as `cursor` with the same filters. `403 changes-off` while Setup **Record Changes** is Off or **Agent Can Read Changes** is off.

26. Error reference

SLUG · STATUS	MEANING	RECOVERY
`invalid-envelope` · 400	Bad or missing `id`, `issuedAt`, `origin`, `data`	Fix the field named in `detail`.
`missing-field` · 400	Required `data.*` absent	Add it.
`invalid-field` · 400	Wrong JSON type or shape; inverted modified window; `SystemModifiedAt` in both window and filter; more than five `languages`; non-object `args`; `args.preview` without `data.preview`; conflicting `target.systemId` and `args.systemId`	Numbers as numbers, booleans as booleans; omit rather than send `null`.
`invalid-filter` · 400	Filter rejected, including a malformed `in()` list (missing or trailing comma, `null` member)	Read `detail`; apply `suggest.field`; rewrite `or` / `not`.
`invalid-cursor` · 400	`data.cursor` does not decode, or belongs to another table or query shape	Pass `@nextLink` unchanged with the same request, or omit the cursor to start over.
`idempotency-key-required` · 400	`invoke` or a mutating `action` without a key while Require Key For Actions is on (previews and non-mutating verbs are exempt)	Add a fresh `idempotencyKey`; reuse it only for retries.
`pk-immutable` · 400	`modify` would change a primary-key field	Insert a new record and delete the old one.
`unknown-field` · 400	Field name not found	Apply `suggest.field`.
`unknown-kind` · 400	Bad `kind`	Use one of `suggest.kinds`.
`unknown-api` · 404	`describe kind:"api"` named something that is not an API page or query	Take the id from `discover kind:"apis"` or a table's `apiRoutes`.
`api-not-routable` · 403 (MCP)	`wicket_api_read` while routing is `Off` or the source table is not readable	Use `query` on `suggest.tableId`.
`unknown-api-field` · 400, `field-restricted` · 403 (MCP)	`wicket_api_read` named a field the API lacks or Wicket does not let the caller read	Use a field from `suggest.fields`, or `query`.
`api-error` · HTTP status (MCP)	Business Central's API refused the read	Read `detail` (Business Central's own message).
`select-and-selectFieldNo` · 400	Both passed	Keep one.
`invalid-select` · 400	`"*"` combined with other entries	Pass `"*"` alone.
`invalid-aggregate` · 400	Bad aggregate shape or combination	Only `count` / `sum` / `avg` / `min` / `max`, only with `filter`.
`invalid-orderby` · 400	Mixed directions or non-indexed field	One direction; indexed fields; or omit.
`record-table-mismatch` · 400	`recordId` belongs to another table	Re-query the right table.
`not-binary` · 400	`readBlob` on a readable non-binary field (named by number)	Pick a `binary: true` field.
`not-exposed` · 403	Table or codeunit not enrolled, or BC permission missing	Add the exposure row or fix permissions.

SLUG · STATUS	MEANING	RECOVERY
`field-masked` · 403	A masked field used in `filter`, `orderBy`, `aggregate`, an effective key or `modifiedAfter` / `modifiedBefore` window, or a masked binary in `readBlob`	Read the field as masked, or filter on a Clear field. An administrator can open it with an exception or the Unmask tier.
`masked-value` · 400	A `modify` sent a masked field's own mask rendering back	Send only the fields you are changing.
`field-restricted` · 403	Field hidden by policy, an `orderBy` whose index also sorts by a hidden field, or a `readBlob` field number that is hidden or does not exist	Do not request it; `describe` shows what is readable.
`trigger-bypass-disabled` · 403	`init: false` or `validate: false` while Allow Trigger Bypass is off	Omit the member so the triggers run; or an administrator allows the bypass.
`replay-withheld` · 403	A resend under an idempotency key whose request **completed**, but whose stored response you may no longer see; members `originalStatus`, `reason`	Do **not** repeat the request under a new key. Restore access (per `reason`) and resend with the same key, or read the current state with a new read.
`invoke-disabled` · 403	`Allow Run` off	Administrator enables it.
`read-only` · 403	Mutating call in Read-Only Mode	Only reads, action previews and non-mutating verbs until lifted.
`unknown-operation` · 404	Operation name not recognised by the dispatcher (a wrong bound-action name normally gets the platform's own HTTP 404 first)	Use one of the operations listed in `suggest`.
`unknown-table` / `unknown-codeunit` · 404	Id does not exist	`discover`.
`unknown-record` · 404	Stale or forged token, or out of scope	Re-query.
`no-page` · 404	No default lookup page	Pass `pageId`.
`unknown-procedure` · 404	Not enrolled	Pick from `suggest.procedures`.
`empty-blob` · 404	Media has no content	Skip.
`etag-required` / `no-etag-available` · 409	Tenant requires an etag	Read first, send `data.etag`.
`batch-too-large` · 400	A `batch` with more than 100 items	Split it.
`invalid-reference` · 400	A batch `fromItem` naming a later item, a deleted record or an unknown field	Reference earlier items only; take the field name from `describe`.
`role-forbidden` · 403	The principal's Wicket role is below the operation	Do not retry; assign the permission set named in `detail`.
`changes-off` · 403	`changes` while the change history is off or not readable by the agent	An administrator turns it on (Setup, Change History); the Wicket Changes page still shows what was recorded.
`changes-forbidden` · 403	`changes` with `user` without `Wicket - Audit Changes`	Omit `user` to see your own changes, or assign the set.
`idempotency-conflict` · 409	Key reused with a different body, also after an earlier attempt released it	New key for a new request.
`idempotency-in-progress` · 409	The same key is still executing	Wait, then resend the identical request for its result.
`idempotency-indeterminate` · 409	An earlier attempt with this key may or may not have taken effect	Query the affected records; retry with a new key if needed.
`precondition-failed` · 412	Stale etag	Re-read, merge, retry.
`payload-too-large` · 413	Filter or select too long	Shorten.
`blob-too-large` · 413	`readBlob` binary over 10 MB	Not readable inline; fetch it another way.
`validation` · 422	Field failed `OnValidate`	Fix the value named in `suggest.field`.
`insert-failed` / `modify-failed` / `delete-failed` · 422	Business Central raised during the write	Read `detail`.
`scan-budget` · 422	Too many rows examined for one page	Tighten the filter, drop `orderBy` to page by key (`suggest.paging`), or deliver asynchronously.
`aggregate-overflow` · 422	The server-side total does not fit the field's type	Narrow the filter, or aggregate in slices and add the results.
`mediaset-unsupported` · 422	`readBlob` on a MediaSet	Read an individual Media field.
`invoke-failed` · 422	Wrapper raised	Read `detail`.
`async-not-configured` · 403	Async off, or the download side incomplete: Stream Proxy selected, Ready SAS without a stored token, or Service SAS without Shared Key upload auth	Read inline; administrator completes the configuration (`detail` names what is missing).
`async-unsupported-op` · 400	`deliver` on a non-query	Only `query` delivers.
`invalid-deliver` · 400	Bad `mode`, `package` or `scheduleAt`	Read `detail`.

SLUG · STATUS	MEANING	RECOVERY
`too-many-requests` · 429	Active-request cap reached	Wait or cancel one; resend with the same key.
`async-start-failed` · 503	The platform could not start a background worker; the request is recorded as failed	Retry shortly.
`async-upload-failed` · (request `problem`)	The package could not be uploaded, or the download configuration became unusable	Read `problem.detail`; resubmit.
`async-task-failed` · (request `problem`)	The scheduled task failed with an unhandled error	Read `problem.detail`; resubmit.
`async-stale` · (request `problem`)	The worker session ended without finishing	Resubmit.
`unknown-request` · 404	Unknown, malformed or foreign request id	Use your own ids.
`unknown-action` · 404	Verb not enrolled	Read `capabilities.actions`.
`action-disabled` · 403	`Allow Run` off	Administrator enables it.
`preview-unsupported` · 400	Preview on a verb without support	Drop `preview`.
`action-failed` · 422	Action raised	Read `detail`.
`skill-not-configured` · 404	No Skill Doc Url	Administrator sets it.
`skill-url-invalid` / `skill-fetch-failed` / `skill-too-large` · 502	Bad or unreachable skill document	Administrator fixes the URL.
`unknown-section` · 404	No such heading	Pick from `suggest.sections`.
`invalid-mode` · 400	Bad `skill` mode	`toc`, `section` or `full`.
`internal` · 500	Unhandled AL error, or an operation whose changes could not be committed (none of them took effect)	Retry reads with backoff; never blindly retry writes (use an `idempotencyKey`: a 5xx leaves an `insert` / `modify` / `delete` key retryable with the same key; an `invoke` / `action` key answers `idempotency-indeterminate`, so verify first).

27. Limits and non-goals

- No events, webhooks or change push: poll with `modifiedAfter`.
- No joins or `$expand`: one table per call; follow `tableRelation` with a second query.
- No SQL passthrough; the filter language is the subset above.
- No bulk writes; one record per call.
- No streaming; page with `@nextLink` or deliver asynchronously.
- No raw binary download from `readBlob`; base64 in JSON, at most 10 MB.
- No reflection; only enrolled wrappers run.
- No request-header semantics; etag and idempotency key travel in the body.

PART V — CONFIGURATION EXAMPLES

Each scenario lists the permission sets, the Wicket configuration and one representative call.

28. Read-only reporting agent

A service principal that reads customers, items and open sales orders for a reporting assistant, and must never write.

Permission sets on the Entra Applications entry: `Wicket - Read` and a customer set granting Read on tables 18, 27, 36 and 37 (or `Wicket Data - Read` if reading anything is acceptable).

Setup: Read-Only Mode on; Mode Exposed tables only; Field Policy Mode `ByClassification` with the fresh-install rules: Customer Content and System Metadata Clear, EUII and EUPI Masked, the rest Hidden. Fields classified EUII or EUPI reach the agent with the `PARTIAL` rendering (a fixed `****` plus the last two characters). To show e-mail as `j****@example.com`, add a **Wicket Field Exposure** row for that field with Read Access Masked and Mask Profile `EMAIL`. No principal needs `Wicket - Unmask`.

Exposure: rows for tables 18, 27, 36, 37 with `Allow Read` on and every other flag off.

Representative call:

JSON

```
{ "id": "rep-1", "issuedAt": "2026-09-04T08:00:00Z", "origin": "reporting-agent",
  "data": { "tableId": 36, "select": ["No.", "Sell-to Customer Name", "Order Date", "Status"],
            "filter": "\"Document Type\" eq 'Order' and Status eq 'Open'",
            "orderBy": ["No. asc"], "top": 100 } }
```

Even if someone later assigned this principal a write-capable data set, the platform would still refuse `insert` because `Wicket - Read` holds no Execute on the write handlers, and Read-Only Mode would refuse it again.

29. Data-entry agent that creates sales orders

An agent that turns e-mails into sales orders: it inserts headers and lines, may correct a line, and never deletes.

Permission sets: `Wicket - Edit` and a customer set with Read on 18 and 27, Read/Insert/Modify on 36 and 37.

Setup: Read-Only Mode off; Require ETag On Writes on; Strong ETag on; Operation Log Retention Days 90.

Exposure:

OBJECT ID	ALLOW READ	ALLOW INSERT	ALLOW MODIFY	ALLOW DELETE
18	on	off	off	off
27	on	off	off	off
36	on	on	on	off
37	on	on	on	off

Representative calls:

JSON

```
{ "id": "so-1", "issuedAt": "2026-09-04T08:10:00Z", "origin": "order-intake", "idempotencyKey": "mail-7f3a-header",
  "data": { "tableId": 36, "values": { "Document Type": "Order", "Sell-to Customer No.": "10000" } } }
```

JSON

```
{ "id": "so-2", "issuedAt": "2026-09-04T08:10:05Z", "origin": "order-intake", "idempotencyKey": "mail-7f3a-line-1",
  "data": { "tableId": 37, "values": { "Document Type": "Order", "Document No.": "SO-1042", "Type": "Item", "No.": "1000", "Quantity": 2 } } }
```

The idempotency keys make a retried e-mail run harmless: the same key replays the original result. A `modify` on the line must carry the `@etag` from the read, or it fails with `409 etag-required`; it cannot change the line's key fields (`400 pk-immutable`). With Allow Trigger Bypass off (the fresh-install default) the agent cannot skip the tables' triggers, so number series and defaults always apply.

30. Posting agent through the Actions companion

An agent that posts released sales orders each evening.

Permission sets: `Wicket - Execute`, `Wicket Act`, `Wicket Data - Read`, and the standard sales posting permission sets your tenant uses.

Setup: Read-Only Mode off.

Catalogs: `Allow Run` on the `post-sales` row of the Action Catalog. Leave `post-purchase` and the journal actions off.

Preview first. Before each post the agent sends the same call with `"preview": true` and no key: one call returns the entries the post would create (or, with `wouldPost: false`, why it would fail), without posting and without consuming a number. It posts only what previewed cleanly, and shows the amounts to a person when they exceed a threshold.

Representative call:

```json
{ "id": "post-1", "issuedAt": "2026-09-04T18:00:00Z", "origin": "evening-posting", "idempotencyKey": "post-SO-1042",
  "data": { "action": "post-sales", "target": { "systemId": "2f4b1c8e-…" }, "args": { "ship": true, "invoice": true } } }
```

Expected result: `{ posted: true, sourceDocumentType: "Order", sourceNo: "SO-1042", postedShipmentNo: "…", postedInvoiceNo: "…" }`. A posting error comes back as `422 action-failed` with Business Central's message in `detail`; a post that committed and then hit an error in later code comes back as `posted: true` with `postCommitWarning`, and must not be posted again. The idempotency key is mandatory on a fresh install (Require Key For Actions) and is what makes a timed-out evening run safe to resend. The generic `insert` / `modify` / `delete` plane is closed to this principal; it can only run the enabled verb.

31. Nightly export to Azure Blob Storage

A data platform pulls all customer ledger entries changed since the previous night.

Permission sets: `Wicket Data - Read`.

Setup: Async Enabled on; Azure Storage Account `contosodata`; Azure Blob Container `wicket-exports`; Azure Auth Mode `Shared Key` with the key stored through **Set Azure Credential**; Download Mode `Service SAS`; Package TTL Hours 48; SAS TTL Minutes 30; Max Active Requests 5.

Exposure: table 21 with `Allow Read` on.

Representative call, submitted at 23:00 for 02:00:

```json
{ "id": "exp-2026-09-05", "issuedAt": "2026-09-04T23:00:00Z", "origin": "data-platform", "idempotencyKey": "cle-2026-09-05",
  "data": { "tableId": 21, "select": ["*"], "modifiedAfter": "2026-09-04T02:00:00Z",
            "deliver": { "mode": "azureBlob", "package": "ndjson", "scheduleAt": "2026-09-05T02:00:00Z" } } }
```

Poll `status` after 02:00 until it reports `succeeded`, then download `packageUrl` (take it from the latest `status` call: the SAS is minted per call). The package stays available for 48 hours after the request finished. The idempotency key guarantees a re-submitted job returns the same `requestId`. Store the `serverTime` from `whoAmI` at submit time as the next night's `modifiedAfter`.

32. Consultant working across several customers with the MCP server

One person, three tenants: a sandbox they own (service principal), a customer's production they may only read (service principal, read-only), and another customer where they sign in as a guest user.

`~/.config/wicket/connections.json`:

```json
{
  "default": "contoso",
  "connections": {
    "contoso": {
      "tenantId": "00000000-1111-2222-3333-444444444444",
      "environment": "ContosoSandbox",
      "companyId": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee",
      "authMode": "client_credentials",
      "clientId": "app-guid-here",
      "keychainService": "wicket",
      "keychainAccount": "contoso",
      "description": "Contoso sandbox, S2S, writable"
    },
    "fabrikam": {
      "tenantId": "55555555-6666-7777-8888-999999999999",
      "environment": "Production",
      "companyId": "ffffffff-0000-1111-2222-333333333333",
      "clientId": "app-guid-here",
      "secretEnv": "FABRIKAM_BC_SECRET",
      "readOnly": true,
      "unmask": false,
      "description": "Fabrikam production, writes refused client-side, masked view only"
    },
    "northwind": {
      "tenantId": "77777777-8888-9999-aaaa-bbbbbbbbbbbb",
      "environment": "Production",
      "companyId": "cccccccc-dddd-eeee-ffff-000000000000",
      "authMode": "device_code",
      "clientId": "public-client-guid-here",
      "description": "Signed in as myself; my BC user permissions apply"
    }
  }
}
```

`"unmask": false` on a connection makes the MCP server add `"unmask": false` to every `query`, `readBlob` and `describe` it sends, so the connection sees the baseline masked view even if its principal holds `Wicket - Unmask` or an exception. It can only lower visibility; there is no setting that asks for more.

In Business Central: the Contoso principal holds `Wicket - Edit` plus a data set; the Fabrikam principal holds `Wicket - Read` plus `Wicket Data - Read`; the consultant's guest user at Northwind holds `Wicket - Read` and whatever data permissions the customer grants. The agent targets a connection per call (`connection: "fabrikam"`) and verifies with `wicket_whoami` after every switch. Handles such as `@recordId`, `@etag` and cursors are connection-specific and must not be reused across connections.

33. Locked-down production

A tenant where Wicket exists for read-only analysis by an internal agent, with an auditor who reviews usage.

Permission sets: the agent's principal holds `Wicket - Read` plus a narrow customer set; the auditor holds `Wicket - Audit`; the operator holds `Wicket - Admin`; log retention runs automatically.

Setup: Read-Only Mode on; Mode Exposed tables only; Field Policy Mode `Explicit`; Describe Restricted Fields off; Async Enabled off; Skill Doc Url set to the hosted guide; Operation Log Retention Days 365.

Field Exposure: rows with Read Access Clear for exactly the fields the analysis needs, for example table 18 fields 1 (No.), 2 (Name), 35 (Country/Region Code) and 58 (Balance (LCY)).

What the agent sees: `describe` on table 18 lists those four fields plus nothing else; `select ["*"]` returns them; naming any other field returns `403 field-restricted` without revealing the field's name. Every call appears on the Operation Log with the agent's `origin`, and the auditor can read the log and the exposure pages without being able to change them.

34. Install and configure

The short path: the quick start covers the plugin install and `/wicket:setup` together with the Business Central side, on one page.

The MCP server, the plugin and the usage skill live in their own repository, davesmith7403/wicket. In Claude Code, install the plugin:

```
/plugin marketplace add davesmith7403/wicket
/plugin install wicket@wicket
```

Then run **`/wicket:setup`**. It asks how you sign in: in a browser window (recommended), with a device code, or as an app with a client secret. It works out the tenant from a pasted Business Central address or a work email, lists the environments and companies to pick from, then saves and tests the connection. No GUID is typed by hand, and no restart is needed. Without a client ID of your own, Wicket uses its default client, so no app registration is needed. A client secret never passes through the chat: setup opens a one-time page in your browser, and the secret goes from there into your computer's own secret store (the Keychain on a Mac, protected for your Windows account on Windows). The same command shows what is configured (`/wicket:setup view`), fixes a failing connection, changes the default and removes connections. The server starts even when nothing is configured, and every tool then answers `503 connection-unavailable` pointing to `/wicket:setup`.

In the Claude desktop app, download `wicket-<version>.mcpb` from the plugin repository's latest release and open it: the app installs Wicket with one click, the same server and skill as the plugin. Then say **"set up Wicket"** in a chat; the same guided setup runs there, and connections are shared with Claude Code on the same machine. Pages Claude builds in the desktop app, such as a dashboard, can query Wicket live while they are open.

To run the server standalone, or to configure connections by hand, clone that repository:

```bash
cd mcp
bun install
cp .env.example .env
```

VARIABLE	REQUIRED	MEANING
`BC_TENANT_ID`	yes	Entra ID tenant GUID
`BC_ENVIRONMENT`	yes	Business Central environment name
`BC_COMPANY_ID`	yes	Company GUID, brace-less
`BC_CLIENT_ID`	yes	Service-principal client ID
`BC_CLIENT_SECRET`	yes	Service-principal secret (prefer `connections.json` with the system's secret store or `secretEnv`)
`BC_BASE_URL`	no	Override of the API host; must be `https://` (plain `http://` only for `localhost`)
`BC_ORIGIN`	no	Custom `origin` tag; default `mcp.wicket/<version>+<connection>`
`BC_READ_ONLY`	no	`1` makes the env connection refuse writes client-side
`WICKET_CONNECTIONS_FILE`	no	Path to `connections.json`; default `~/.config/wicket/connections.json`

Register with Claude Code:

```bash
claude mcp add wicket -- bun run /absolute/path/to/wicket/mcp/src/index.ts
```

Or in Claude Desktop's `claude_desktop_config.json`:

```json
{ "mcpServers": { "wicket": { "command": "bun", "args": ["run", "/absolute/path/to/wicket/mcp/src/index.ts"] } } }
```

Secrets resolve per connection in this order: the system's secret store (`keychainService` + `keychainAccount` : the Keychain on macOS, a DPAPI-protected file on Windows, stored through `/wicket:setup`), an environment variable named by `secretEnv` , or an inline `clientSecret` for development only. The server validates the whole file at start. It sets an entry aside, with its reason, for any of these: an unknown key, a missing secret source, a braced GUID, a tenant that is not a GUID or a domain, an environment name outside `[A-Za-z0-9_-]` , or a `baseUrl` that is not https (the bearer token travels on every request). The other entries still load. `wicket_connections` lists the set-aside entries under `rejected` .

A `device_code` or `browser` connection never blocks a tool call on the human. The first call answers a `401 sign-in-required` problem: for device code it carries the verification URL and code (`verificationUri` , `userCode`); for browser sign-in a window has opened. The sign-in keeps running in the background, and the next call after the user signs in picks up the token. On a `401` from Business Central the server mints a fresh token and retries once.

35. Tools

TOOL	WICKET OPERATION	NOTES
`wicket_discover`	`discover`	Accepts `publisher` (an AL filter on the owning app's publisher, e.g. `"<>Microsoft"` for solution tables); rows carry `app` .
`wicket_describe`	`describe` , cached for tables	A table's answer is cached per connection and company on disk and revalidated on every call with `ifNoneMatch` , so a policy change is never served stale; a cached answer says `fromCache: true` . `summary: true` returns a compact view (primary key, keys, field names, types, access and relations, the owning app, `referencedBy`) without captions. `referencedBy: true` asks for the reverse relations.
`wicket_map`	local	The tenant map: one Markdown file per connection (`~/.config/wicket/maps/<connection>.md`), one `##` section per table or topic. `read` reports `stale: true` when `capabilities.structureVersion` has moved since the last write; `upsert` replaces or appends one section; `remove` drops one. Structure only, never record values: the file sits outside Wicket's field policy, so a value written there escapes masking. 128 KB cap.
`wicket_query` , `wicket_read_blob`	the read plane	`wicket_query` accepts `aggregate` and `modifiedAfter` / `modifiedBefore` ; it is read-only and refuses a `deliver` block with `400 use-deliver-tool` .
`wicket_query_deliver`	`query` with `deliver`	Async export to Azure Blob Storage: the `wicket_query` inputs plus a required `deliver` block and an optional `idempotencyKey` (a resubmit then replays to the same `requestId`). Not read-only, but allowed on `readOnly` connections.
`wicket_query_all`	`query` looped	Follows `@nextLink` up to `maxPages` (default 100) or `maxRows` (default 5 000, at most 50 000; the walk stops after the page that reaches it); flags `@truncated` with the resume cursor; returns partial results with a resume cursor on a mid-run failure. No `aggregate` , no `deliver` .
`wicket_apis`	`discover` with `kind: "apis"` , cached	The tenant's API catalog, cached per environment in memory and in `~/.config/wicket/cache/` (or `$WICKET_CACHE_DIR`) and revalidated with one `ifNoneMatch` call. Without `tableId` / `search` / `group` it answers a summary by route prefix.
`wicket_api_read`	`describe` with `kind: "api"` , then a GET on the API	Reads through one API page after a fresh governance check; see chapter 19. Not recorded in the Operation Log; the server writes one stderr line per read.
`wicket_insert` , `wicket_modify` , `wicket_delete` , `wicket_invoke` , `wicket_action`	the write and execute planes	Accept `idempotencyKey` (required by Wicket for `invoke` and `action` while Require Key For Actions is on). When an `invoke` or `action` call carries none, the server mints one and returns it as `@idempotencyKey` , so a call whose response was lost can be retried with the same key instead of posting twice, whatever the tenant's switch says. Marked destructive through MCP tool annotations so hosts prompt before running them.
`wicket_capabilities` , `wicket_page_url` , `wicket_whoami`	self-description, deep link, identity	
`wicket_status` , `wicket_cancel` , `wicket_requests`	async lifecycle	The server does not poll; the agent does.
`wicket_connections` , `wicket_use`	local	List connections; switch the session's active one. Prefer the per-call `connection` parameter.
`wicket_setup`	local, Entra and Business Central's own APIs	Guided setup behind `/wicket:setup` . Actions: `status` , `locate` , `signin` , `environments` , `companies` , `save` , `remove` , `setDefault` . A saved connection works at once.
`wicket_skill`	local	The bundled usage guide — SKILL.md plus reference files (tools, filters, errors, writes, presenting, automation, tables, solutions, protocol) — with `toc` / `section` (a section or a whole file) / `full` modes.

The server sends the skill's essentials as MCP `instructions` at the handshake, so a host such as Claude Code places them in the session's system prompt before the first call; every tool carries a title and read-only/destructive annotations; a page of rows is rendered one row per line to keep results small. In Claude Code the simplest install is the `wicket` plugin (chapter 34), which brings the server, the skill and the `/wicket:setup` , `/wicket:orient` and `/wicket:inventory` commands in one

step. Without it, the skill (`skills/wicket-usage/` in the plugin repository) is installed in Claude Code by placing it under `~/.claude/skills/` or `.claude/skills/` , or uploaded to claude.ai as a zip under Customize › Skills, from where it also syncs into Claude Code and Cowork.

Every BC-touching tool accepts `connection: "<name>"` . A problem document is returned as an MCP tool error with the full RFC 7807 body, so the agent sees `detail` and `suggest` . A connection marked `readOnly: true` refuses the five mutating tools with `403 connection-read-only` before a token is minted; this complements, and does not replace, Wicket's Read-Only Mode and the permission sets.

When a call's answer is lost (a timeout, or a 5xx from the gateway), the server's problem says what to do next according to the call: a read is safe to retry; a write sent with an `idempotencyKey` is resent with the same key (it replays, or answers `idempotency-in-progress` / `idempotency-indeterminate`); a write sent without one must not be blindly retried: query the affected records first.

A. Object map

AL object names keep the `Bcmag` prefix from the product's former name: renaming a table renames its SQL table and an upgrade would lose the data. Captions, permission sets and the API route use the Wicket name.

TYPE	ID	NAME	PURPOSE
Page (API)	94100	Bcmag Rpc Endpoint	The single HTTP surface, `api/wicket/rpc/v1.0`, entity `wicketEndpoint`
Table	94100	Bcmag Setup	Per-company configuration singleton
Table	94101	Bcmag Exposure	The exposure list: one row per table or codeunit
Table	94102	Bcmag Operation Log	One row per request
Table	94103	Bcmag Idempotency Log	Replay snapshots
Table	94104	Bcmag Procedure Catalog	Invoke enrolment
Table	94105	Bcmag Field Exposure	Field policy overrides
Table	94106	Bcmag Request Queue	Async delivery requests
Table	94107	Bcmag Action Catalog	Named actions
Table	94108	Bcmag Execute Gate	Empty role-gate table; write rights granted from `Wicket - Execute` up
Table	94109	Bcmag Write Gate	Empty role-gate table; write rights granted from `Wicket - Edit` up
Table	94110	Bcmag Idempotency Reservation	One row per caller and key: the at-most-once execution reservation
Table	94111	Bcmag Classification Rule	Read verdict and mask profile per data classification
Table	94112	Bcmag Mask Profile	How a masked value is rendered
Table	94113	Bcmag Access Exception	Per-user verdict overrides with an optional window
Table	94114	Bcmag Unmask Gate	Empty sentinel table; write rights granted only by `Wicket - Unmask`
Page	94101	Bcmag Setup Card	Setup
Page	94102	Bcmag Exposure List	Exposure
Page	94103	Bcmag Operation Log List	Audit
Page	94104	Bcmag Procedure Catalog List	Invoke enrolment
Page	94105	Bcmag Field Exposure List	Field overrides
Page	94106	Bcmag Request Queue List	Async requests, with Cancel
Page	94107	Bcmag Secret Input	Masked secret dialog
Page	94108	Bcmag Action Catalog List	Actions
Page	94109–94111	Bcmag Classification Rules, Mask Profiles, Access Exceptions	Field policy rules, profiles and exceptions
Codeunit	94100	Bcmag Rpc Dispatcher	Envelope, idempotency key rules, replay, async fork, audit, telemetry
Codeunit	94101–94107	Bcmag Envelope, Problem, Logger, Field Name Resolver, Type Mapper, Projector, Record Id Codec	Envelope parsing, problem documents, logging, projection
Codeunit	94108–94110	Bcmag Filter Lexer, Parser, Applier	The OData filter subset to parameterised `SetFilter`
Codeunit	94125, 94126	Bcmag Op Runner, Op Runner Context	The `Codeunit.Run` boundary: the gates (Read-Only Mode, then the role rung), then routing to one handler
Codeunit	94111–94119, 94138, 94140–94144	Bcmag Op …	One handler per operation (discover … capabilities, status, requests, action, skill, pageUrl, whoAmI)
Codeunit	94139	Bcmag Async Cancel	The `cancel` handler and the queue page's Cancel action
Codeunit	94130	Bcmag Op Aggregate	Server-side aggregates for `query`
Codeunit	94120, 94128	Bcmag Install, Bcmag Upgrade	Setup defaults, reaper arming, upgrade backfill

TYPE	ID	NAME	PURPOSE
Codeunit	94121	Bcmag Log Purge	Retention job for logs, snapshots and reservations
Codeunit	94122–94124, 94127, 94129	Value Coercer, Invoke Context, Caption Resolver, Record Access, Field Policy	Shared services of the handlers
Codeunit	94131–94137	Bcmag Azure Client, Sas Signer, Package Builder, Async Enqueuer, Worker, Failure, Reaper	Async delivery
Codeunit	94145	Bcmag Json Read	Typed JSON member reads
Codeunit	94146	Bcmag Role Gate	The role-rung check on the gate tables
Codeunit	94147	Bcmag Idempotency Guard	Takes, settles and judges reservations
Codeunit	94148	Bcmag Op Authorizer	The gates shared by the runner and replay; re-authorizes stored responses
Codeunit	94150–94152	Bcmag Policy Defaults, Field Masker, Hash Key	Shipped field-policy posture, mask rendering, the tenant HMAC key
Codeunit	94153	Bcmag Async Reaper Guard	Reaper task failure codeunit; re-arms the chain (also run on every enqueue)
Codeunit	94154	Bcmag Async Queue Purge	Retention purge of finished queue rows
Codeunit	94155	Bcmag Async Enqueue Run	Error boundary around the enqueue body
Codeunit	94190	Bcmag Wrap Ping	Sample invoke wrapper
Codeunit	94157	Bcmag Action Rollback	The forced-rollback boundary for previews and non-mutating verbs
Codeunit (companion)	94200, 94202, 94203, 94204	Bcmag Act Post …	Posting wrappers
Codeunit (companion)	94207, 94208	Bcmag Act Preview, Bcmag Act Preview Capture	Posting previews on Business Central's preview engine
Codeunit (companion)	94209	Bcmag Act Doc Resolver	Document addressing for the newer wrappers
Codeunit (companion)	94211–94214	Bcmag Act Release/Reopen Sales/Purch Doc	Release and reopen wrappers
Codeunit (companion)	94215–94217	Bcmag Act Sales Quote To Order / To Inv, Purch Quote To Order	Quote conversion wrappers
Codeunit (companion)	94218, 94219	Bcmag Act Document Pdf, Bcmag Act Item Availability	The non-mutating render and compute wrappers
Permission set	94102–94107	Wicket - Objects / Read / Execute / Edit / Audit / Admin	The ladder
Permission set	94110, 94111	Wicket Data - Read / Wicket Data - Exec	Generic data sets
Permission set	94112	Wicket - Unmask	The unmask tier
Permission set (companion)	94200	Wicket Act	Wrapper reachability

B. Operation by permission set

OPERATION	READ	EXECUTE	EDIT	ADMIN	DATA - READ	DATA - EXECUTE
discover, describe, query, readBlob, capabilities, skill, pageUrl, whoAmI	yes	yes	yes	yes	yes	yes
status, requests, cancel, async deliver	yes	yes	yes	yes	yes	yes
invoke, action	no	yes	yes	yes	no	yes
insert, modify, delete	no	no	yes	yes	no	no
Admin pages	no	no	no	yes (Audit views)	no	no
Set Azure Credential, Log Purge	no	no	no	yes	no	no

C. Common Business Central table ids

ID	TABLE	ID	TABLE
18	Customer	112	Sales Invoice Header
23	Vendor	113	Sales Invoice Line
27	Item	114	Sales Cr.Memo Header
36	Sales Header	115	Sales Cr.Memo Line
37	Sales Line	122	Purch. Inv. Header
38	Purchase Header	123	Purch. Inv. Line
39	Purchase Line	270	Bank Account
81	Gen. Journal Line	5050	Contact
17	G/L Entry	5740	Transfer Header
21	Cust. Ledger Entry	5741	Transfer Line
25	Vendor Ledger Entry	5600	Fixed Asset
32	Item Ledger Entry	79	Company Information
110	Sales Shipment Header	111	Sales Shipment Line

Confirm fields with `describe` before writing; for vertical solutions use `discover` with a caption filter rather than guessing ids.

D. Glossary

TERM	MEANING
Bound action	An OData operation attached to an entity, invoked as `POST …/entity(key)/Microsoft.NAV.name`. Wicket's operations are bound actions on the `wicketEndpoint` singleton.
Envelope	The JSON body every call carries: `id`, `issuedAt`, `origin`, optional `idempotencyKey`, `data`.
Exposure	The row that makes a table or codeunit reachable.
Field policy	The per-field read/write decision Wicket adds on top of Business Central: Hidden, Masked or Clear for reading.
Masked	A field verdict: the value is returned as a mask rendering and cannot be filtered, sorted, aggregated or written back.
Unmask tier	The `Wicket - Unmask` permission set, which lifts Masked to Clear for its holder.
Problem document	The RFC 7807 error body: `type`, `title`, `status`, `detail`, `instance`, `operation`, `suggest`.
Wrapper	A small AL codeunit enrolled in the Procedure Catalog that performs one business operation on behalf of `invoke` or `action`.
Package	The NDJSON, JSON or ZIP file an async delivery writes to Azure Blob Storage.
Reaper	The self-re-arming maintenance task that fails stale async workers, expires delivered packages, retries unconfirmed blob deletions and purges old queue rows.
Principal	The identity a call runs as: a service principal (application) or a delegated user.
Rung	One level of the permission-set ladder: Read, Execute, Edit.